

Privacy Lens: An Evidence-Bounded Android Privacy Review Framework with Replaceable Regulation Packs (Revision 10)

Zhiheng Zhang

Student ID: 54560484

Supervisor: Richard Sinnott

August 2026

Abstract

This thesis presents Privacy Lens, a local-first Android prototype that converts bounded permission-audit summaries into explainable prompts for human privacy review. The research addresses a recurring problem in mobile privacy tooling: technical observations are frequently presented with more authority than their acquisition scope, semantics, or legal relevance can support. Privacy Lens therefore treats provenance, temporal scope, missing context, and uncertainty as first-class data. Its compliance engine rejects malformed or unsafe records, isolates overlapping evidence windows, preserves synthetic-source labels, and records the exact regulation pack and rule version applied to every finding. The evaluated legal objective is the European Union General Data Protection Regulation, but jurisdiction-specific mappings are loaded through a replaceable pack contract rather than embedded in the Android, storage, or interface layers.

The implemented product uses a three-destination information architecture comprising Overview, Findings, and Settings. It stores findings on the device, requests no sensitive runtime permission, disables Android backup, and provides explicit controls for clearing local evidence. Evaluation separates deterministic rule conformance, adversarial boundary behaviour, package integration, manifest facts, and physical-device rendering. A fixed-seed 1,000-case campaign produced 800 true positives, 200 true negatives, and no observed disagreement with the specified oracle; extended tests covered malformed inputs, temporal overlap, replay, source isolation, and regulation-pack identity. Release APK and Android App Bundle builds targeting SDK 36 succeeded, and the release APK installed and launched on an OPPO PERM00 device. These results establish a bounded engineering candidate, not legal compliance, unrestricted Android visibility, population accuracy, accessibility conformance, or application-store approval. The principal contribution is a claim-to-evidence discipline that is maintained consistently across architecture, interface, evaluation, and deployment documentation.

Contents

1	Introduction	6
1.1	Problem Context	6
1.2	Why Mobile Permission Evidence Matters	6
1.3	The Evidence-to-Decision Gap	7
1.4	Consent, Attention, and Interface Burden	8
1.5	Accountability Rather Than Automated Adjudication	8
1.6	Regional Extensibility as a Governance Problem	9
1.7	Research Method	9
1.8	Research Questions	10
1.9	Objectives and Contributions	10
1.10	Scope and Claim Boundary	10
1.11	Thesis Structure	11
2	Literature Review and Foundations	11
2.1	Regulatory Principles and Technical Signals	11
2.1.1	From Legal Principle to Review Question	11
2.1.2	Data Protection by Design and Interface Conduct	12
2.2	Android Visibility and Permission Semantics	12
2.2.1	A Taxonomy of Mobile Privacy Evidence	13
2.2.2	Historical Operations and the Ordinary-App Boundary	13
2.3	Human Factors and Calibrated Trust	14
2.3.1	Cognitive Load and Progressive Disclosure	14
2.3.2	Trust Calibration and Explanation Quality	15
2.3.3	Accessibility as an Evidence Requirement	15
2.4	Replaceable Rule Systems	15
2.4.1	Pack Lifecycle and Review Governance	16
2.4.2	Rule-Based and Learned Decision Models	16
2.4.3	Automated Validation and Oracle Independence	17
2.5	Store Delivery as a Trust Boundary	17
2.6	Synthesis	17
3	Requirements and System Design	17
3.1	Design Position	17
3.2	Requirements	18
3.3	Evidence Contract	18
3.4	Regulation-Pack Contract	19
3.5	Layered Architecture	19
3.6	Threat Model	19

3.7	Design Alternatives and Rationale	19
3.8	End-to-End Decision Lifecycle	21
3.9	Failure-State Design	22
3.10	Data-Minimisation Decisions	23
3.11	Requirement Priorities and Release Gates	24
3.12	Stakeholders and Usage Scenarios	25
3.13	Functional Decomposition	25
3.14	Non-Functional Requirements	26
3.15	Evidence Schema and Provenance	26
3.16	Admission and Fail-Closed Validation	27
3.17	Temporal Isolation Model	28
3.18	Rule-Pack Contract in Detail	28
3.19	State, Persistence, and Migration	29
3.20	Security and Abuse Cases	29
3.21	Traceability from Requirement to Evidence	29
3.22	Evaluation Strategy	30
3.23	Design Invariants	30
3.24	Decision Records for Future Extension	31
4	Implementation	31
4.1	Application Identity and Delivery Configuration	31
4.2	Rule-Pack Engine	32
4.3	Validation and Temporal Isolation	32
4.4	Native Android Boundary	32
4.5	Information Architecture and Visual Design	32
4.6	Privacy and User-Control Implementation	33
4.7	Codebase Reconciliation	33
4.8	Technology Stack and Module Boundaries	33
4.9	Regulation-Pack Implementation	34
4.10	Runtime Validation Pipeline	34
4.11	Signal Computation	35
4.12	Finding Construction and Communication	36
4.13	Repository Behaviour	36
4.14	Android Bridge and Scheduling	36
4.15	Interface Implementation	37
4.16	Brand and Accessibility Implementation	37
4.17	Build, Permission, and Signing Configuration	38
4.18	Implementation Summary	38

5	Evaluation and Results	38
5.1	Evidence Model	38
5.2	Source and Rule Verification	39
5.3	Release Build and Manifest Verification	39
5.4	Physical-Device Evaluation	39
5.5	Usability and Trust Assessment	40
5.6	Evaluation Questions and Acceptance Logic	40
5.7	Reproducibility Controls	41
5.8	Deterministic Corpus Interpretation	42
5.9	Adversarial Input Results	42
5.10	Temporal and Replay Results	43
5.11	Accountability and Presentation Results	43
5.12	Regulation-Pack Results	44
5.13	Build Reproduction	44
5.14	Manifest and Installed-Package Reconciliation	44
5.15	Physical Interaction Protocol	45
5.16	Visual Review Method	45
5.17	Result Boundary	46
5.18	Evidence Triangulation	46
5.19	Oracle Independence and Correlated Defect Risk	47
5.20	Metric Interpretation and Error Analysis	48
5.21	Store-Candidate Evidence Review	49
5.22	Evaluation Repeatability and Change Control	49
6	Discussion and Limitations	50
6.1	What the Prototype Establishes	50
6.2	Construct and Internal Validity	51
6.3	External and Ecological Validity	51
6.4	Legal and Regional Validity	51
6.5	Usability and Accessibility	52
6.6	Interpreting the Engineering Result	52
6.7	Acquisition Authority and Observation Quality	53
6.8	Threshold Meaning, Calibration, and Decision Error	54
6.9	Temporal Evidence and Replay Semantics	55
6.10	Regulation-Pack Governance Beyond Software Modularity	56
6.11	Trust, Comprehension, and Willingness to Use	57
6.12	Privacy and Security of the Reviewing Application	58
6.13	Research Ethics and Responsible Evaluation	59
6.14	Store and Operational Readiness	60

6.15	Prioritised Next Work	60
7	Conclusion	60
7.1	Answers to the Research Questions	60
7.2	How the Contributions Fit Together	62
7.3	Contributions	63
7.4	Final Product Position	63
7.5	Deployment and Evidence Roadmap	64
7.6	Lessons for Privacy-Engineering Practice	65
7.7	Future Work	66
7.8	Closing Statement	66

1 Introduction

1.1 Problem Context

Mobile applications ask users to make privacy decisions while the evidence needed to evaluate those decisions remains fragmented. A permission declaration states capability, not necessarily use; a runtime observation can indicate activity, but not purpose, lawful basis, necessity, or proportionality. This distinction is especially important for health-related applications, where personal data can fall within the GDPR’s special-category regime. The Regulation requires legal and organisational analysis in addition to technical safeguards, including the principles in Article 5 and data protection by design and by default in Article 25 [1,2].

Privacy Lens addresses a narrower engineering problem: how can an ordinary Android application present available permission-audit evidence without transforming incomplete telemetry into a false legal conclusion? The application collects no remote evidence, has no account or advertising layer, and stores findings locally. It exposes whether a finding came from device-visible observations or a synthetic demonstration, displays missing-evidence limitations, and prepares questions for human review.

The work began as a GDPR-oriented prototype. Revision 9 retains GDPR as the evaluated legal objective but removes GDPR from the core decision mechanism. Legal mappings are loaded through replaceable regulation packs. This separation makes regional extension possible while preventing the interface, repository, and Android bridge from silently inheriting one jurisdiction’s terminology.

1.2 Why Mobile Permission Evidence Matters

Mobile privacy is difficult to reason about because a handheld device combines intimate data, continuous availability, and many independent software actors. A user may encounter an application as a single icon, yet that application can depend on operating-system services, embedded libraries, cloud endpoints, analytics components, advertising identifiers, and background tasks. The apparent simplicity of a permission prompt therefore hides a larger processing system. The prompt may describe access to a sensor or data category, but it does not normally explain every purpose, recipient, retention period, inference, or downstream combination. Contextual integrity theory is useful here because it treats privacy as the appropriateness of information flows within a social context rather than as secrecy alone [3]. A technically permitted flow can still be surprising or inappropriate when its purpose, recipient, or persistence differs from the context understood by the user.

Health-related applications make that mismatch especially consequential. Digital health systems may process symptoms, medication routines, reproductive information, movement patterns, sleep, location, or communications. The IQVIA review of digital-health development describes a field moving toward more specialised and evidence-oriented interventions [4]. Parker and colleagues, studying Android health applications, found extensive data-sharing and transparency problems [5]. These studies do not imply that every permission access is harmful, nor that every health application is governed identically. They establish that

mobile-health privacy cannot be reduced to a one-time installation choice and that users, developers, researchers, and reviewers need better evidence about what a system can observe.

Android’s permission model provides necessary protection, but permission state and processing purpose are different constructs. A granted permission indicates that an application may invoke a protected capability under specified platform conditions. It does not state whether the capability was invoked, why an invocation occurred, whether derived data were retained, or whether the processing satisfied a legal obligation. Conversely, a denied or absent permission does not prove the absence of all related data because an application may receive information through another service, user input, previously collected records, or a less sensitive signal from which attributes can be inferred. The prototype therefore treats permission observations as one evidence family within a broader review, not as a complete representation of processing.

1.3 The Evidence-to-Decision Gap

The central research problem can be described as an evidence-to-decision gap. At one end of the gap are platform facts: a package identifier, permission category, access count, observation window, acquisition source, and timestamp. At the other end are questions that matter to a person or regulator: Was processing expected? Was it necessary for the stated purpose? Was a valid lawful basis available? Were special-category conditions satisfied? Were recipients and retention disclosed? Was the data subject able to exercise meaningful control? No deterministic transformation can recover answers that were never present in the input.

Two common design errors follow from ignoring this gap. The first is semantic inflation, in which a technical event is labelled as a legal violation. This gives a small signal the authority of a much broader assessment. The second is evidential silence, in which an empty or unavailable audit is displayed as a reassuring green state. On stock Android, ordinary applications often lack authority to inspect unrestricted cross-application history. Treating an empty result as absence of activity would therefore reward the system precisely when its visibility is weakest.

Privacy Lens introduces an intermediate output: a review finding. A finding can say that the available count crossed a research threshold, that evidence is unavailable, or that no technical concern was identified within the observed window. It can also state which contextual facts are missing. This format preserves utility without claiming certainty. The distinction is comparable to the difference between a screening instrument and a diagnosis: screening can identify cases that deserve attention, but its result must be interpreted in context and by an appropriately qualified actor.

The design also recognises that explanation is not an optional layer added after computation. Explanation begins with data modelling. If the repository stores only a severity colour and a message, the interface cannot later reconstruct the source, rule version, temporal scope, or missing context. Revision 10 consequently defines provenance fields as part of the decision object. The interface is a view of that object rather than a separate narrative that can drift away from it.

1.4 Consent, Attention, and Interface Burden

Privacy interfaces operate under severe attention constraints. McDonald and Cranor quantified the impractical time cost of reading the privacy policies encountered by an ordinary user [6]. Obar and Oeldorf-Hirsch demonstrated how readily people accept online terms without substantive engagement [7]. These findings should not be interpreted as evidence that users are careless. They show that notice-and-choice systems frequently demand more time and legal comprehension than a routine interaction can support.

A warning application can reproduce this failure if it presents a wall of legal text, asks the user to interpret raw telemetry, or interrupts every time a count changes. More information is not automatically more usable transparency. Information must be staged according to the decision a user can reasonably make. Privacy Lens therefore begins with three questions: what requires attention, what evidence is missing, and what can be done next. Rule references and technical details remain available, but they follow the status and action rather than preceding them.

This approach also affects notification strategy. Repeated alerts for an unchanged state can teach users to ignore the product. A review system should distinguish new evidence, a material escalation, and a stable finding. Although the present candidate does not claim a longitudinal notification study, its repository retains enough state to support transition-based notification in future work. The architectural principle is that attention is a limited resource and should be consumed only when evidence meaningfully changes.

1.5 Accountability Rather Than Automated Adjudication

The GDPR’s accountability principle requires a controller not only to comply with the Regulation but also to be able to demonstrate compliance [1]. A consumer-facing application cannot fulfil the controller’s entire accountability obligation, because it does not possess the controller’s records of processing, contracts, purposes, retention decisions, risk assessments, or organisational measures. It can nevertheless support accountability by making a narrow technical observation reproducible and by showing which questions remain unanswered.

This positioning changes the desired failure mode. A conventional classifier may optimise for a single label and treat rejected input as an implementation inconvenience. An accountability tool should prefer an explicit hold or evidence gap to a falsely reassuring result. Unsafe types, unrecognised sources, impossible windows, and missing context therefore fail closed. The system does not invent a value to keep the pipeline moving. This behaviour is especially important at a native bridge, where JavaScript assumptions can be violated by malformed, stale, or unexpected platform data.

Accountability also requires version knowledge. A finding that says only GDPR is not reproducible if rules or interpretations later change. Privacy Lens records a pack identifier and version, an official source, and a particular rule reference. Historical findings are not silently reinterpreted when the active pack changes. This creates a minimal audit trail: a reviewer can identify which data were available, which pack was applied, and why a particular message appeared.

1.6 Regional Extensibility as a Governance Problem

The request for regional switching creates both an architectural opportunity and a governance risk. It is straightforward to define a common software interface for multiple packs. It is much harder to ensure that each pack represents its jurisdiction accurately. Legal systems differ in scope, terminology, actors, exemptions, enforcement, and the relationship between national and sectoral rules. A design that assumes every jurisdiction can be translated into GDPR article equivalents would merely hide legal coupling behind a new file format.

Revision 10 therefore separates software extensibility from legal validity. The registry proves that the application can discover, select, persist, and apply different pack objects. It does not certify the content of every possible pack. The EU GDPR pack is the evaluated legal objective. The Research Baseline pack is deliberately non-legal and demonstrates the mechanism without suggesting that another jurisdiction has been reviewed. A future regional pack must identify its authority, effective date, scope, official source, reviewer, localisation status, fixtures, migration policy, and limitations.

This distinction is central to the thesis contribution. Decoupling is valuable not because it makes law interchangeable, but because it makes jurisdiction-dependent assumptions visible and reviewable. When a legal mapping is isolated, it can be versioned, challenged, replaced, and tested without changing the Android bridge or product shell. The architecture therefore supports expansion while making the cost of responsible expansion explicit.

1.7 Research Method

The work follows an iterative design-and-evaluation method. First, legal and platform constraints are translated into claim boundaries. Second, these boundaries become data and interface requirements. Third, the requirements are implemented across TypeScript, React Native, Kotlin, storage, and Android build configuration. Fourth, each layer is evaluated with evidence appropriate to that layer. Finally, contradictions between intended claims and observed behaviour are used to revise both code and thesis.

The method combines constructive and empirical elements. The constructive element is the artefact: the rule-pack contract, compliance engine, repository, Android bridge, and interface. The empirical element consists of deterministic test vectors, adversarial boundary cases, build outputs, merged-manifest inspection, installed-package inspection, ADB interaction, UI hierarchies, screenshots, and filtered runtime logs. The method does not include a controlled participant experiment or independent legal assessment; those omissions are recorded as limitations rather than replaced with speculative conclusions.

An important methodological rule is that evidence is not pooled across incompatible layers. A thousand logical cases do not become a thousand device trials. A successful installation does not validate every WorkManager schedule. A well-rendered warning does not prove comprehension. A rule reference does not prove infringement. Maintaining these separations allows the final evaluation to be positive about what worked without obscuring what remains unknown.

1.8 Research Questions

The thesis answers four questions:

1. How can Android permission-audit signals be transformed into deterministic, explainable review findings while unsafe or incomplete inputs fail closed?
2. How can evidence provenance, temporal scope, and missing legal context be communicated so that a technical warning is not mistaken for a legal verdict?
3. How can jurisdiction-specific rules be isolated behind a stable regulation-pack contract without weakening traceability?
4. What claims are supported by deterministic tests, release builds, and bounded physical-device evaluation, and which claims remain unverified?

1.9 Objectives and Contributions

The first objective is a reproducible decision pipeline. Every finding retains its permission category, observed count, time window, source, regulation-pack identifier, rule reference, risk level, rationale, and missing-context fields. The second objective is calibrated communication. The interface distinguishes *needs review*, *evidence gap*, and *no technical concern*; none is labelled compliant or non-compliant. The third objective is architectural replaceability. A registry loads immutable rule-pack definitions and the compliance engine consumes their shared contract. The fourth objective is deployable engineering: a stable Android package name, target SDK 36, release APK and Android App Bundle, explicit privacy policy, owner-action checklist, branded assets, and physical-device evidence.

The contribution is therefore not an automated GDPR judge. It is an accountability-support prototype that makes a bounded technical inference reconstructable and leaves legal conclusions to an appropriately authorised human reviewer.

1.10 Scope and Claim Boundary

The evaluated data path covers permission summary records available to the application or generated by its controlled simulator. Stock Android restricts ordinary applications from reading unrestricted activity belonging to other applications. Absence of an observation is consequently an evidence gap, not proof that an access did not occur. The current rule thresholds are research baselines; they are not statutory limits. The included EU GDPR pack is a traceable mapping aid, not legal advice.

Evaluation covers source compilation, deterministic logical tests, adversarial boundary tests, release assembly, manifest inspection, installation, launch, interaction, and short-run observation on one OPPO PERM00 device. It does not establish multi-OEM compatibility, 24-hour scheduler reliability, battery performance, population accuracy, accessibility conformance, independent legal validity, or application-store approval.

1.11 Thesis Structure

Chapter 2 establishes the legal, Android, HCI, and software-architecture foundations. Chapter 3 derives requirements and the evidence-bounded architecture. Chapter 4 documents the implemented application and regulation-pack mechanism. Chapter 5 reports reproducible evaluation evidence. Chapter 6 analyses validity, usability, deployment, and legal limitations. Chapter 7 answers the research questions and defines the next validation programme.

2 Literature Review and Foundations

2.1 Regulatory Principles and Technical Signals

The GDPR regulates processing rather than permission counts. Article 5 establishes principles including lawfulness, transparency, purpose limitation, data minimisation, accuracy, storage limitation, integrity, confidentiality, and accountability. Articles 6 and 9 concern lawful bases and special-category data; Article 25 requires data protection by design and by default [1]. EDPB guidance describes Article 25 as a continuing obligation implemented through appropriate technical and organisational measures [2]. A technical observation may support an accountability inquiry, but cannot by itself demonstrate whether these legal conditions are satisfied.

This distinction determines the framework’s vocabulary. A threshold exceedance is a *review signal*. Missing purpose, lawful-basis, necessity, data-subject, controller, retention, and sharing evidence is shown explicitly. The UI never derives a finding labelled “GDPR violation” or “compliant”. Regulation-pack references provide a reason to investigate and a link to an official source; they do not replace legal interpretation.

2.1.1 From Legal Principle to Review Question

Operationalisation is necessary in any regulatory technology, but it must preserve the semantic distance between a principle and a sensor. Article 5(1)(a) concerns lawfulness, fairness, and transparency. A permission count can contribute to transparency by revealing a pattern that was previously invisible, yet it cannot establish lawfulness or fairness. Article 5(1)(b) concerns purpose limitation. A count contains no purpose unless purpose is supplied as contextual evidence. Article 5(1)(c) concerns data minimisation. Frequency can be relevant to necessity, but a high frequency may be required for continuous navigation while a single collection may be excessive for an unrelated purpose. Article 5(2) concerns accountability. An audit record can support demonstration, but it is only one component of the records, decisions, safeguards, and governance required from a controller [1].

Articles 6 and 9 further show why a permission signal cannot become a verdict. Article 6 provides several possible lawful bases; consent is not automatically the only basis. Article 9 establishes a prohibition and exceptions for special categories, but not every Android dangerous permission maps directly to special-category processing. Location, contacts, microphone, and camera data are context-dependent. Audio might contain health information, ordinary

conversation, environmental noise, or no retained personal data. A technically observed invocation does not identify which of those states occurred.

The pack therefore maps a signal to a sequence of questions. For a location-related pattern, the reviewer may ask what purpose was declared, whether continuous precision was necessary, whether a less intrusive source was available, whether the application was in the foreground, whether location was retained or transmitted, and whether the user could reasonably anticipate the flow. For a microphone-related pattern, the reviewer may ask whether recording was user-initiated, how activation was signalled, whether raw audio left the device, and how long any derivative was retained. These questions are not answers; their value is that they prevent the visible count from crowding out legally decisive facts.

2.1.2 Data Protection by Design and Interface Conduct

Article 25 directs controllers to implement appropriate measures and privacy-protective defaults, taking account of state of the art, cost, context, and risk. EDPB guidance emphasises that this is a continuing obligation rather than a certification achieved once [2]. For Privacy Lens, data protection by design affects both internal processing and the claims made to users. Local storage, disabled backup, no analytics, no advertising, no account, explicit deletion, and minimised permissions reduce the application’s own data footprint. Fail-closed validation and immutable pack metadata reduce the risk that incomplete evidence becomes a misleading result.

Interface conduct is also relevant. A privacy tool could employ alarming colours, default users into sharing diagnostic data, hide evidence limitations, or imply that purchasing a service resolves risk. EDPB guidance on deceptive design patterns provides a broader reminder that formal information can still be presented in a manipulative way [8]. Privacy Lens uses restrained severity language, places limitations near the result, and keeps the demonstration optional. The implementation is not claimed to have been audited against every deceptive-design criterion, but the guidance informs the design objective of calibrated rather than maximised reliance.

2.2 Android Visibility and Permission Semantics

Android permissions are part of a layered security model. Manifest declarations, runtime grants, AppOps state, operating-system services, and OEM behaviour do not expose one universal event stream to an ordinary third-party application. Android’s permission guidance emphasises minimising permission requests and using platform-supported workflows [9]. Therefore, a credible review tool must state its acquisition authority and must not equate an empty result with absence of processing.

Static manifest analysis and dynamic monitoring answer different questions. Static tools can identify declared capabilities and embedded components; dynamic or taint-based systems can observe selected flows but may require instrumentation, system modification, a VPN, or elevated authority. PiOS illustrates the value and complexity of static data-flow analysis [10]. Privacy Lens deliberately occupies a lighter position: it evaluates bounded audit summaries and communicates their coverage rather than claiming universal enforcement or surveillance.

2.2.1 A Taxonomy of Mobile Privacy Evidence

Existing approaches can be organised by the claim their evidence supports. Manifest analysis answers what capabilities an application declares and which components are packaged. It is reproducible and can be performed without executing the application, but it cannot establish that a declared capability was exercised. Static data-flow analysis attempts to connect sources and sinks in program code. It can reveal possible flows beyond manifest declarations, although reflection, native code, dynamic loading, and environment-specific paths complicate coverage. PiOS and related systems demonstrate the analytical value of tracing potential information flows while also illustrating that such analysis is specialised rather than a routine end-user feature [10].

Runtime instrumentation answers which monitored operations occurred during a particular execution. Its result depends on exercised paths, instrumentation placement, operating-system authority, and the definition of an event. Network interception answers which traffic crossed an observed interface and may expose destinations or payload structure. It cannot see purely local processing and may be limited by encryption, certificate pinning, protocol choice, and traffic that bypasses the observation path. Platform privacy dashboards can offer authoritative information for selected permissions, but their presentation and export capabilities are determined by the operating system.

Enforcement systems answer yet another question: whether a protected operation was blocked or modified under a policy. Enforcement is stronger than observation for the operation under control, but it can require device-owner status, system integration, rooting, a custom operating system, or application rewriting. These deployment choices change the threat model. A VPN monitor, for example, gains visibility by becoming an intermediary and must itself be trusted. A rooted monitor can inspect more state but weakens assumptions that apply to ordinary consumer devices.

Privacy Lens does not attempt to dominate this taxonomy. It accepts a structured summary from a bounded source and focuses on interpretation, traceability, and communication. This makes it complementary to manifest scanners, platform dashboards, enterprise management systems, and forensic monitors. A future authorised deployment could supply richer evidence through the same decision contract, but the application should continue to display the authority and coverage of that source.

2.2.2 Historical Operations and the Ordinary-App Boundary

The existence of an Android API is not evidence that any application can use it for any package. Access control can depend on signature permissions, usage-access grants, AppOps policy, device-owner status, process identity, Android release, and OEM modification. Historical-operation methods may be public in an SDK while useful cross-package results remain restricted in ordinary deployment. The physical-device evidence collected for this thesis exposed application-owned modes and scheduling state; it did not demonstrate unrestricted enumeration of other packages.

This boundary affects evaluation design. A complete system has an acquisition layer and a

decision layer. The acquisition layer determines what the operating system or an authorised integration provides. The decision layer validates, aggregates, maps, stores, and presents that evidence. Synthetic injection can test the second layer exhaustively without proving the first. Conversely, an acquired record can demonstrate that a bridge works without proving the correctness of every decision boundary. The thesis reports these layers separately.

2.3 Human Factors and Calibrated Trust

Privacy notices impose substantial reading and comprehension costs [6, 7]. Adding another dense dashboard can reproduce the problem it intends to solve. Trustworthy warning design should therefore support quick orientation while preserving access to rationale and provenance. ISO 9241-11 frames usability through effectiveness, efficiency, and satisfaction in a specified context of use [11]. WCAG 2.2 and Android accessibility guidance further motivate non-colour status cues, meaningful labels, and sufficiently large touch targets [12, 13].

These sources support four design choices. First, the overview leads with the current evidence boundary. Second, colour is paired with iconography and text. Third, details are progressively disclosed through finding cards. Fourth, destructive actions such as clearing local findings are separated from routine navigation. These are conformance-oriented design decisions, not proof of usability; participant studies remain necessary.

2.3.1 Cognitive Load and Progressive Disclosure

Cognitive Load Theory distinguishes load inherent to a task from load introduced by its presentation [14]. Privacy review has substantial intrinsic complexity because it combines technical events, purposes, law, expectations, and uncertainty. A product cannot remove that complexity, but it can avoid adding unnecessary navigation, decoration, or jargon. The earlier prototype mixed health dashboards, research projects, experiment controls, scoring views, and privacy findings. That structure made the user’s immediate task unclear. The three-destination Revision 10 architecture reduces extraneous choice and uses a stable pattern for each finding.

Progressive disclosure is not equivalent to hiding caveats. The most important caveat—the limited reach of evidence—appears on the overview before the user initiates a review. Detail cards then reveal the observed count, source, pack, reference, rationale, and missing context. This ordering allows rapid orientation while preserving auditability. A legal citation without an explanation would be too terse; a full reproduction of statutory text on every card would be too heavy. The interface therefore supplies a short reference and offers the official source separately.

Dual-process accounts distinguish fast, heuristic judgement from slower deliberation [15]. Severity colour can produce a rapid response, which is useful for prioritisation but dangerous if colour appears to communicate legal certainty. Textual labels, source chips, explicit caveats, and neutral action language are intended to interrupt an overly simple red-equals-illegal inference. This intention requires empirical testing. The thesis does not infer comprehension from the presence of these elements.

2.3.2 Trust Calibration and Explanation Quality

Trust is calibrated when reliance corresponds to actual capability. A system that understates uncertainty can induce over-reliance; a system that only displays caveats may become unusable and be ignored. Lee and See describe the importance of appropriate reliance on automation [16]. In Privacy Lens, calibrated trust is pursued through claim symmetry: a positive-looking result and a warning must both show their evidence basis. A no-technical-concern state means that no rule fired within the admitted evidence, not that processing was lawful. A needs-review state means that a rule fired, not that a violation occurred.

Explanation quality has at least four dimensions. Fidelity asks whether the explanation corresponds to the implemented rule. Completeness asks whether the important inputs and limits are represented. Comprehensibility asks whether the intended audience can understand it. Actionability asks whether the explanation supports a reasonable next step. The deterministic rule and stored pack metadata support fidelity. Missing-context questions support completeness. The card hierarchy aims at comprehensibility. Suggested review questions support actionability. Only the first two dimensions are strongly evaluated in the present work; comprehension and actionability need participant evidence.

2.3.3 Accessibility as an Evidence Requirement

Accessibility affects whether transparency is available in practice. WCAG 2.2 requires that information is not conveyed through colour alone and includes criteria concerning contrast and target size [12]. Android guidance recommends meaningful content descriptions and interaction targets that are large enough to operate reliably [13]. The implementation pairs icons and words with colour and supplies accessibility labels for primary controls.

These measures are design evidence, not conformance certification. True accessibility evaluation should include screen-reader order, announcements after asynchronous review, large-font reflow, switch access, keyboard focus, high-contrast settings, reduced motion, colour-vision deficiencies, and comprehension with users who rely on assistive technology. The distinction mirrors the wider thesis: source inspection can establish that a label exists, whereas a user study is needed to establish that the resulting interaction is effective.

2.4 Replaceable Rule Systems

Hard-coding statutory names into a decision engine creates maintenance and validity risks. A legal text may change, another jurisdiction may use different concepts, and a translated label may not preserve the same meaning. The appropriate software pattern is a stable engine contract with versioned, independently reviewable rule data. A pack should declare its jurisdiction, version, official source, caveat, supported signals, thresholds, references, and explanatory text. The engine should reject unknown packs and preserve the selected pack identifier in every finding.

Replaceability does not mean equivalence. Two packs cannot be treated as interchangeable simply because they implement the same TypeScript interface. Each requires independent

legal review, localisation, validation fixtures, and governance over updates. The included “Research Baseline” pack demonstrates switching without representing a second jurisdictional legal interpretation.

2.4.1 Pack Lifecycle and Review Governance

A credible pack lifecycle begins before implementation. The maintainer identifies the intended jurisdiction, material and territorial scope, regulated actors, effective date, official consolidated source, language versions, and the technical signals actually available. A qualified reviewer then maps signals to questions, not verdicts, and records assumptions and exclusions. Engineering converts the reviewed mapping into immutable data and fixtures. Localisation is reviewed separately because legally important distinctions can be lost even when the code remains valid.

Release governance should treat a pack as a versioned dependency. A substantive legal amendment, authoritative interpretation, corrected translation, changed threshold, or altered missing-context question may require a new pack version. Findings retain the version used at evaluation time. Updating the active pack should not rewrite historical records. If a migration is needed, it should create a new evaluation linked to the old one rather than silently changing the original rationale.

Pack selection also raises a scope question. Device location alone is not a reliable selector because applicable law can depend on establishment, targeting, residence, processing context, contracts, and sector. Revision 10 therefore requires explicit user selection from registered packs and presents the jurisdiction label and caveat. Automatic selection could be added only with a defensible scope model and an override.

2.4.2 Rule-Based and Learned Decision Models

Rule-based systems are attractive for a legal-support interface because a reviewer can reconstruct the condition that fired. A fixed threshold also has obvious limitations: it can be bypassed by splitting activity across windows, may not represent purpose, and may treat different application categories alike. Learned anomaly detection can adapt to patterns, but it introduces training-data bias, distribution shift, opacity, and uncertainty. A hybrid system could combine transparent guardrails with contextual or personalised models, provided that model provenance and uncertainty remain visible.

The present engine intentionally uses explicit research thresholds. The contribution lies in safe admission, temporal isolation, versioning, explanation, and boundary disclosure rather than in claiming optimal thresholds. Population calibration would require representative applications, ground-truth definitions, stratification by purpose and category, independent annotation, and validation on data not used to choose parameters. Without that evidence, a mathematically sophisticated adaptive threshold could be less trustworthy than a simple rule whose limits are stated.

2.4.3 Automated Validation and Oracle Independence

Randomised tests can explore more input combinations than a short hand-written suite, but their evidential value depends on the generator and oracle. Uniform random counts may seldom exercise exact thresholds. A generator that only creates well-typed values cannot test the runtime boundary. An oracle copied from the production helper may reproduce the same defect. The evaluation therefore combines fixed seeds, boundary values, malformed types, permission mutation, temporal constructions, and an explicit expected-result calculation.

Fuzzing and property-oriented testing are especially useful for invariants. Counts should remain finite and non-negative. Results should not depend on the order of unrelated package histories. Replaying an identical window should not increase totals. Overlapping aggregate windows should not be added. A simulator source should never become a native-observation source. Unknown packs should not load. These properties express safety behaviour more clearly than a single accuracy number.

Statistical language must also match the sample. A deterministic 1,000-case run provides strong regression coverage for its generator, but it is not a random sample of real applications or alleged infringements. Confidence intervals applied to such a run would describe the generated distribution, not the mobile ecosystem. Revision 10 reports the confusion matrix descriptively and reserves population claims for future independent datasets.

2.5 Store Delivery as a Trust Boundary

Application-store readiness includes more than a successful APK. Google Play uses Android App Bundles for new application delivery, requires current target API levels, and requires developers to complete a Data safety declaration [17–19]. Signing identity, a public privacy-policy URL, support contact, screenshots, listing text, and console review remain owner-controlled steps. A research build can be an initial submission candidate while still being unpublishable until these external obligations are completed.

2.6 Synthesis

The literature yields four gaps addressed by the prototype: permission signals are easily over-interpreted; acquisition coverage is often hidden; legal mappings are commonly coupled to product code; and engineering success is frequently conflated with deployment or legal validity. The following chapter converts these gaps into explicit system requirements and evidence contracts.

3 Requirements and System Design

3.1 Design Position

Privacy Lens is designed as a local review instrument. Its output is a structured prompt for investigation, not a legal decision and not a security-enforcement action. The system accepts bounded permission summaries, validates their shape and temporal relationships, applies the

active regulation pack, stores the resulting findings locally, and presents the result with its provenance and limitations.

3.2 Requirements

Table 1 defines the requirements used for implementation and evaluation.

Table 1: Revision 9 requirements and acceptance evidence

ID	Requirement	Evidence
F1	Evaluate supported permission summaries deterministically.	Seeded and boundary suites
F2	Preserve source, time window, pack, rule, and missing context.	Type and repository tests
F3	Reject unsafe identifiers, numbers, sources, timestamps, windows, and contexts.	Adversarial tests
F4	Switch only among registered, versioned regulation packs.	Pack registry tests and Settings UI
F5	Separate synthetic demonstrations from device-visible evidence.	Source labels and simulator tests
U1	Provide overview, findings, and settings through stable navigation.	Physical screenshots and UI trees
U2	Communicate severity without colour alone and disclose evidence limits.	Source inspection and physical rendering
P1	Keep findings local, disable Android backup, and request no sensitive runtime permission.	Manifest and installed package inspection
D1	Build a versioned release APK and AAB with a stable package identity.	Release build and device install

3.3 Evidence Contract

An input record contains package identifier, permission category, count, start and end timestamps, source, and contextual metadata. Validation occurs before rule evaluation. Identifiers must match conservative formats; counts and timestamps must be finite safe integers; the window must be ordered; the source must be enumerated; and simulator-labelled records cannot be promoted to observed-device evidence. Invalid inputs produce a review-hold result rather than a permissive “no concern” result.

Temporal aggregation is bounded by package, permission, and non-overlapping window. Exact replay replaces the previous record, overlapping windows are excluded, adjacent completed windows may be aggregated, retention follows the latest observed end time, and history is capped. These rules reduce double counting and memory-exhaustion risk but cannot reconstruct event-level truth from aggregate windows.

3.4 Regulation-Pack Contract

A regulation pack is immutable application data with the following elements: pack identifier, display name, jurisdiction label, version, official source URL, legal caveat, supported permission rules, thresholds, risk level, reference label, rationale template, and missing-context questions. A registry exposes only known packs. The generic engine receives the pack explicitly and writes the pack identifier and version into each finding. Consequently, historical findings remain interpretable if the active selection later changes.

The initial registry contains an EU GDPR accountability-review pack and a non-legal Research Baseline pack. The latter proves architectural switching only. A future regional pack must be treated as a governed artefact: obtain qualified interpretation, record legal version and effective date, add jurisdiction-specific fixtures, review translations, and define migration behaviour.

3.5 Layered Architecture

The system has five layers:

1. the Android bridge and WorkManager components expose bounded audit and simulator capabilities;
2. validation and temporal isolation establish admissible evidence;
3. the rule-pack engine produces explainable findings;
4. the repository stores findings and persisted pack selection in app-private storage;
5. the React Native interface renders overview, findings, and settings with explicit claim boundaries.

The dependency direction is inward: interface and bridge code do not implement legal thresholds, and rule packs do not access Android or storage. This structure supports testing the engine independently and replacing a pack without reconstructing the application shell.

3.6 Threat Model

The model includes malformed bridge values, forged simulator provenance, unsafe numbers, replayed windows, overlapping summaries, unbounded history, misleading empty results, and user over-reliance on authoritative language. It does not defend against a compromised operating system, a maliciously modified application binary, falsified system telemetry, or a legally incorrect pack supplied by a trusted maintainer. Those risks require platform integrity, signing governance, and independent legal review.

3.7 Design Alternatives and Rationale

Several alternative product positions were considered. The first was a binary compliance scanner that would display compliant or non-compliant for every application. This was rejected because

permission frequency cannot establish purpose, necessity, lawful basis, controller obligations, or applicable jurisdiction. A binary label would hide missing evidence and encourage users to treat a technical heuristic as a legal decision. The adopted three-state model retains a needs-review state, an evidence-gap state, and a no-technical-concern state whose wording is explicitly limited.

The second alternative was a single risk score. A score appears compact and supports sorting, but it combines unlike uncertainties. A high aggregate count, missing purpose, synthetic source, and consent contradiction do not have a common natural unit. Weighting them would introduce assumptions that are difficult to justify and easy to overlook. Privacy Lens instead stores contributing signals and renders a reasoned state. A future prioritisation score could be added for professional triage only if every contribution remains inspectable and calibration evidence exists.

The third alternative was a learned classifier trained on application labels or permission histories. Machine learning could model interactions not captured by fixed thresholds, yet the project lacked an independently labelled, representative, legally meaningful data set. Training on store categories, malware labels, or developer declarations would answer different questions. A deterministic rule engine was selected because it makes boundary behaviour, missing inputs, and change history reviewable. This choice does not claim that rules are universally superior; it matches the evidence available for the thesis.

The fourth alternative was to embed GDPR language in interface components. This is initially convenient because each screen can display the relevant article directly. It creates duplication, complicates review, and makes regional extension risky: changing jurisdiction would require editing presentation and perhaps persistence logic throughout the application. The pack contract centralises legally motivated content and forces the engine to record which pack generated a finding. Generic screens display pack-provided labels through validated fields.

The fifth alternative was to choose a region automatically from device locale or location. Locale does not reliably determine applicable law, organisational establishment, data-subject status, or processing context. Automatic selection could therefore present the wrong framework with unwarranted confidence. The adopted design requires explicit user selection from registered packs and shows the selected jurisdiction and caveat. It does not collect location to decide which privacy law to discuss.

The sixth alternative was a cloud account with central evidence storage. Centralisation could simplify synchronisation, remote dashboards, and model updates, but it would also create a new repository of sensitive behavioural metadata and require authentication, server security, retention, international transfer analysis, and deletion workflows. The current product is local first, uses no account, and offers local deletion. This choice reduces deployment features but aligns the application's conduct with its privacy objective.

The seventh alternative was to ingest unrestricted free-form pack files. This would maximise extensibility but expose users to misleading sources, malformed thresholds, pack confusion, and potentially executable content. The present registry admits bundled immutable definitions. Future signed declarative distribution may be appropriate, but only after schema, publisher,

rollback, and review controls exist.

The eighth alternative was to treat absence of observations as zero activity. This would produce a cleaner dashboard, but it would conflate a real zero with unavailable acquisition. The selected model requires a capability-aware source and reports an evidence gap when coverage is missing. This is less reassuring and more accurate.

The ninth alternative concerned notifications. Emitting an alert for every evaluation would demonstrate activity but create repetition and fatigue. Repository logic instead considers whether a finding is new, has changed state, or has escalated. Silent storage remains possible for unchanged or low-priority outcomes. The design can later support controlled comparison of notification policies without changing decision semantics.

The final alternative concerned storage of raw observations. Keeping a complete event history would improve forensic detail but increase sensitivity and resource cost. The candidate stores bounded findings and aggregate history needed for current temporal reasoning. Event-level acquisition is deferred until an authority model, minimisation policy, and retention design can be evaluated. This is a deliberate privacy-performance trade-off rather than an accidental omission.

3.8 End-to-End Decision Lifecycle

The lifecycle begins when the user initiates review, a scheduled worker requests work, or the controlled simulator generates an example. The acquisition component returns candidate records together with a source identity. It must not return a native zero merely because a platform capability is unavailable. Capability failure and an observed zero are different states and lead to different user decisions.

Each candidate crosses the admission boundary as unknown data. The validator checks the package identifier, permission key, count, window, timestamps, source, and optional context without relying on compile-time types. Rejection is structured. The record does not enter temporal history and cannot contribute to a reassuring status. The caller can present the failure as an evidence problem and retain enough diagnostic detail to correct the producer without exposing raw sensitive content unnecessarily.

An admitted record is placed in a key space defined by package and permission. Exact-window replay replaces the prior aggregate. Independent completed windows may be retained for cross-window analysis; overlapping aggregates are not summed. Simulator data remains excluded from accumulated cross-window claims. Retention and a per-key cap bound state. These operations prepare evidence before jurisdictional rules are applied.

The engine receives the selected immutable pack explicitly. It locates the supported rule, calculates daily excess, evaluates burst information where timestamps are available, examines compatible history, and inspects any admitted processing context. The result includes the observed and threshold values, signal types, status, communication priority, rationale, legal or research reference, missing-context questions, and pack identity. The calculation never mutates the pack.

The repository compares the result with the stored finding for the same logical subject. It

distinguishes an unchanged state, a new concern, resolution or reversal, and risk escalation. This comparison governs notification eligibility. It also writes a bounded slice under the versioned repository key. A finding remains attached to its original pack even if Settings later selects another pack.

Projection turns repository records into interface-facing models. It calculates overview totals by state, applies Findings filters, formats source and temporal labels, and exposes actions. Projection should not infer new legal semantics. If a required field is absent, it retains an unknown or evidence-gap presentation. This makes projection testable independently of React components.

Presentation then orders information according to user need. The overview states reach and limitation before the main action. Findings allow scanning by state and inspection of reasons. Settings places pack choice, caveat, simulator, privacy information, and deletion controls in separate groups. Accessibility labels name the action and state rather than depending on icon or colour. Destructive action requires deliberate confirmation where implemented.

The lifecycle ends with user action or explicit non-action. A user may seek a privacy notice, ask a developer for purpose and retention information, repeat review with authorised evidence, or decide that the technical signal does not require immediate attention. Privacy Lens does not contact the reviewed application, change its permissions, or submit a complaint automatically. Keeping the final decision outside the engine preserves human accountability.

Deletion is also part of the lifecycle. The user can clear local findings, and uninstall or platform data clearing removes the application data within Android's boundary because backup is disabled. Export or future synchronisation would create additional copies and must define deletion separately. The current lifecycle intentionally ends at local storage.

3.9 Failure-State Design

Failure states were designed as product states rather than exception messages. Acquisition unavailable means the current deployment cannot obtain suitable records; it is not displayed as a completed scan. Invalid evidence means a producer supplied data that fails the contract; it cannot be silently repaired into a normal case. Unknown pack means evaluation is blocked until a registered definition is selected; it does not fall back to the EU pack merely because that pack is bundled.

Storage failure also requires conservative behaviour. If persisted data cannot be parsed or migrated safely, the interface should not mix it with current findings. Recoverable records may be retained after field-level validation, while unrecoverable content should be quarantined or omitted with a visible explanation. Clearing data remains available so the user can recover control. Production logging should report the failure class without printing evidence payloads.

Work scheduling failure is distinct from review failure. WorkManager can defer periodic work because of platform policy, Doze, reboot, or vendor restrictions. The interface should present the last completed review time and source capability rather than imply continuous monitoring. A registered request demonstrates scheduling intent, not on-time execution. Future long-run tests should measure delay and recovery before stronger wording is used.

Partial evaluation is another possibility. Some records may be valid while others are rejected. Treating the batch as wholly clean would hide rejection; treating it as wholly failed would discard useful evidence. A suitable summary reports accepted findings together with the count and reasons for excluded records. The current bounded flows establish structured rejection at record level; richer batch disclosure remains an extension point.

Pack-content failure has two forms. Schema failure is machine detectable and should block admission. Substantive legal error may only be identified by expert review or later correction. Pack provenance, versions, official sources, and historical identity allow the maintainer to issue a successor without pretending earlier findings never existed. The interface can mark a deprecated pack and recommend re-evaluation while preserving the original record.

Network failure is intentionally limited because core review and bundled packs operate locally. Privacy-policy and official-source links may be unavailable, but the application should retain the relevant title and caveat. If remote pack delivery or synchronisation is later added, offline operation and last-known-good behaviour become acceptance requirements rather than optional resilience features.

Visual failure includes clipped localisation, unreadable contrast, focus loss, hidden caveats, and controls obscured by system insets or keyboards. These failures may not appear in logic tests. Rendered screenshot matrices, hierarchy inspection, screen-reader use, font scaling, and device variation are therefore part of the planned evidence. The one-device acceptance check establishes only the candidate baseline.

Finally, misunderstanding is treated as a failure even when the software behaves as coded. If users interpret no technical concern as GDPR compliance, or mistake simulator records for device evidence, the product has failed its communication objective. Participant comprehension and reliance tests are needed to detect that class. The design reduces the risk through terminology and provenance, but does not declare it eliminated.

3.10 Data-Minimisation Decisions

The application needs enough evidence to explain a review signal, yet every retained field has privacy cost. Package identifier, permission family, count, time window, source, pack identity, and decision reason form the current minimum for reconstructing an aggregate finding. Optional context is admitted only when supplied for the review purpose. The application does not require account identity, contact lists, precise location content, microphone content, or advertising identifiers.

Counts are stored instead of raw sensor values. A location access finding does not require coordinates, and a microphone access finding does not require audio. This distinction is central: the system reviews evidence that an operation occurred, not the personal data produced by the operation. A future connector should preserve that separation unless a separately approved research question requires content.

Time resolution is also minimised. Aggregate start and end values support window validity, replay, and bounded temporal analysis. Timestamp arrays are accepted for controlled burst evidence, but they should not be collected by default merely because the schema can represent

them. The source capability and purpose determine whether additional resolution is justified.

Local-first storage avoids creating a server-side behavioural database. The trade-off is that the user or controlled deployment is responsible for device access and loss, and professional collaboration features are limited. The design disables Android backup so evidence is not restored unexpectedly to another device through ordinary platform backup. This choice should be re-evaluated if enterprise-managed backup becomes a stated requirement.

Bounded history limits denial-of-service and retention exposure. The cap is not a substitute for purpose-based deletion, but it prevents indefinite growth under current aggregate use. Repository slices and clear controls make the lifecycle understandable. Future event-level storage would require stronger indexing, encryption assessment, retention schedules, and selective deletion.

Notifications contain only the minimum text needed to indicate a changed review state. Detailed evidence remains inside the application. Lock-screen visibility can still expose the existence of a concern, so future production settings should consider notification privacy and user preference. The current evaluation does not claim that every device lock-screen configuration was tested.

External links are used for official materials and privacy documentation instead of embedding broad web content. The application does not send the finding to those sites. Users should be informed that opening a link transfers them to another controller and may reveal ordinary browser metadata. Remote analytics or link tracking would contradict the minimisation posture unless separately justified.

Data minimisation finally applies to evaluation artefacts. Screenshots, UI dumps, log excerpts, and device metadata should avoid unrelated notifications, account names, and installed-application lists. Synthetic demonstrations are preferred for public evidence. Physical records used for thesis acceptance remain scoped to the test application and bounded device session.

3.11 Requirement Priorities and Release Gates

Requirements were prioritised according to the harm caused by failure. Evidence admission, provenance, non-verdict language, source separation, replay safety, pack identity, and deletion are release blockers because failure could create misleading privacy conclusions or remove user control. Core navigation, readable hierarchy, package identity, release builds, and absence of sensitive runtime permissions are also blockers for the initial store candidate.

Important but non-blocking research extensions include multi-OEM acquisition measurement, long-duration WorkManager observation, full screen-reader matrices, participant studies, production signing, and independent legal review. They are non-blocking only for the bounded engineering candidate. They become blockers for stronger claims such as public production readiness, accessibility conformance, or validated GDPR guidance.

Optional features include cloud synchronisation, accounts, remote pack updates, application blocking, and learned ranking. These features do not repair current evidence gaps automatically and would introduce substantial security, privacy, or governance work. They should be

considered only after the core assurance chain is stable.

The release gate follows dependency order. A source-level failure blocks package evaluation because the binary would not represent an accepted program. A package or manifest failure blocks device acceptance. A device crash or broken primary journey blocks visual promotion. A missing legal or usability study does not turn an engineering check red; it keeps the associated substantive claim explicitly unverified.

This priority model avoids two extremes. It does not delay all prototype learning until every production activity is complete, and it does not allow a successful demonstration to erase unresolved authority or legal questions. Each gate authorises a named next activity. The Revision 10 candidate is designed to authorise controlled store-submission preparation, internal-track testing, and participant evaluation.

3.12 Stakeholders and Usage Scenarios

The primary user is a person who wants to understand available privacy evidence without becoming an Android or GDPR specialist. This user needs a short account of what was observed, why it deserves attention, what is not known, and what action is reasonable. A second stakeholder is a developer or product owner using the application during internal review. This stakeholder needs reproducible rule identifiers, pack versions, timestamps, and evidence-source labels. A third stakeholder is a privacy or legal reviewer who may use the finding as an entry point into a broader assessment. This stakeholder needs the official source, missing-context questions, and an explicit statement that the application has not reached a legal conclusion.

The first usage scenario is an ordinary review with no prior data. The overview explains the scope before offering a review action. If the bridge returns no admissible observations, the interface reports an evidence gap rather than a clean bill of health. The second scenario is a populated review. Findings are grouped by status and can be inspected individually. The third scenario is learning through synthetic data. The user runs a controlled demonstration, which creates a varied but deterministic set of records marked as synthetic. The fourth scenario is changing legal perspective. The user selects another registered pack, sees its jurisdiction and caveat, and performs a new review without relabelling historical findings. The fifth scenario is withdrawal from the tool itself: the user clears local findings or uninstalls the application.

These scenarios exclude unsupported uses. Privacy Lens is not intended to block another application, intercept network traffic, prove that a sensor callback stopped, obtain evidence by evading platform restrictions, or submit a legal complaint automatically. It does not select applicable law from device location. Defining exclusions is important because a polished interface could otherwise suggest a broader product than the evidence pipeline implements.

3.13 Functional Decomposition

The review workflow is decomposed into acquisition, admission, temporal preparation, evaluation, persistence, projection, and presentation. Acquisition produces a candidate summary and a capability statement. Admission checks whether the record can be trusted as a well-formed

input. Temporal preparation determines whether history can be combined without double counting. Evaluation applies one immutable pack. Persistence writes findings and migration metadata. Projection converts stored findings into dashboard counts and sections. Presentation renders this projection with actions and caveats.

This decomposition allows each failure to be represented at the correct layer. If the bridge is unavailable, the application should not create a zero-count native record. If a timestamp is unsafe, the engine should not try to repair it silently. If two aggregate windows overlap, the repository should not sum them. If a pack identifier is unknown, the registry should not fall back to whichever pack appears first. If storage migration fails, the interface should avoid presenting stale records as current evidence. The design favours explicit absence over invented continuity.

The simulator follows the same downstream path after acquisition. It is not a separate UI mock that bypasses validation. This is important because a demonstration that constructs presentation objects directly would not test the engine, repository, or projection. Synthetic source identity is introduced at acquisition and persists through every later stage.

3.14 Non-Functional Requirements

Security requirements include fail-closed input handling, conservative identifier formats, bounded history, source enumeration, immutable pack definitions, disabled backup, and no sensitive runtime permissions. Privacy requirements include local processing, no remote evidence service, no analytics or advertising SDK, and a clear deletion control. Reliability requirements include deterministic results for identical admitted inputs, replay replacement, pack-specific persistence, and migration from the previous repository schema.

Usability requirements include a clear initial action, a stable three-destination navigation model, non-colour status semantics, concise summaries, optional detail, meaningful accessibility labels, and separation of destructive controls. Maintainability requirements include one authoritative compliance path, typed interfaces, registry-based packs, reusable theme constants, tests for new packs, and removal of duplicate legacy services. Deployability requirements include a stable application identifier, explicit version, current target SDK, release APK and AAB generation, upload-key configuration, privacy-policy content, and documented owner actions.

Performance is treated as a constraint rather than a validated claim. The decision engine operates on bounded in-memory arrays and simple rule lookup. Storage is local and capped per package-permission key. WorkManager is selected for deferrable periodic work rather than exact scheduling. No quantitative battery, long-duration memory, or startup objective is claimed in Revision 10 because the available evidence does not cover those dimensions broadly enough.

3.15 Evidence Schema and Provenance

The evidence schema is designed around reconstructability. A permission summary includes a package identifier, supported permission key, non-negative safe-integer count, start and end

of the aggregate window, source enumeration, and optional processing context. The source is one of the controlled values supported by the application, such as native bridge, imported evidence, or simulator. A free-form source string is rejected because it could allow synthetic data to impersonate device evidence.

Context fields represent what a legal review would need but a permission count cannot provide. Examples include purpose, controller identity, lawful basis, special-category condition, foreground relationship, consent state, retention statement, and sharing information. Their absence does not make the technical record invalid. It changes the finding by adding missing-context questions and preventing a legal conclusion. This distinction separates data-quality failure from legal incompleteness.

A finding contains the admitted evidence plus decision metadata: status, risk, rule identifier, threshold, rationale, regulation-pack identifier and version, official source, evaluation time, and missing-context fields. The schema is intentionally verbose. Storage efficiency is less important than the ability to explain why a result appeared and which assumptions applied.

Provenance also constrains metrics. Simulator precision and recall are computed only for records whose expected labels are part of the controlled generator. Imported or native records do not receive an accuracy label because their legal and behavioural ground truth is unknown. Aggregating these sources into one accuracy metric would create a number with no coherent denominator.

3.16 Admission and Fail-Closed Validation

Validation occurs at runtime even though the application is written in TypeScript. Static types protect code paths compiled from known sources, but values crossing a native bridge, storage migration, import boundary, or untyped caller can violate those types. The validator therefore treats each field as unknown until checked.

Counts and timestamps must be finite JavaScript safe integers. This excludes negative values, fractional event counts, infinities, not-a-number values, and integers whose arithmetic cannot be represented exactly. The time window must have a strictly earlier start and a later end. An evaluation timestamp must be consistent with the window. Retention values must be positive and bounded. Package identifiers and permission keys must match conservative patterns rather than merely being non-empty strings.

Context validation follows the same rule. String fields must actually be strings before trimming; booleans must not be inferred from truthful text or numbers. The engine does not coerce a value such as the text `false` into a Boolean because this can reverse meaning. Unknown source and permission values produce an invalid or review-hold state instead of falling through to a default threshold.

Fail-closed does not mean the application crashes. It means the system returns a safe, explicit result that prevents incomplete evidence from being described as no concern. The UI can then explain which field was rejected or why review cannot proceed. This approach supports both security and diagnosability.

3.17 Temporal Isolation Model

Aggregate windows create a double-counting problem. Suppose two records report counts for intervals that overlap. Without event identifiers, the system cannot know whether events in the shared interval appear in both counts. Summing them could manufacture a cross-window threshold. Revision 10 therefore aggregates only completed, non-overlapping windows for the same package and permission.

For windows $w_i = [s_i, e_i)$ and $w_j = [s_j, e_j)$, compatibility is defined as

$$e_i \leq s_j \quad \vee \quad e_j \leq s_i. \quad (1)$$

Adjacency is allowed because half-open intervals that meet at one boundary do not overlap. Exact replay is detected by equal package, permission, start, and end. The newer replay replaces the stored value rather than increasing the history size or total. This makes processing idempotent for a repeated aggregate export.

Retention uses a monotonic watermark derived from the latest admitted window end for the key. Basing expiry on a newly arrived record's timestamp alone would allow a late or forged old record to move the horizon backwards. Records older than the bounded horizon relative to the watermark are removed. Each package-permission history is also capped at a finite maximum. The cap is a denial-of-service safeguard, not a claim about the maximum legitimate event rate.

Cross-window detection is consequently conservative. It may fail to combine useful but overlapping aggregates, yet it avoids asserting a total that the evidence does not support. Event-level identifiers or disjoint platform buckets would permit a richer model. The current design states the limitation rather than guessing how much overlap occurred.

3.18 Rule-Pack Contract in Detail

The common pack contract separates descriptive metadata from executable rule data. Descriptive metadata includes identifier, display name, jurisdiction label, version, official source, effective or review date, and caveat. Rule data binds a supported permission signal to a research baseline, threshold logic, severity, rationale, reference label, and missing-context questions.

The registry is the only route from a persisted identifier to a pack object. This prevents arbitrary downloaded or user-entered text from being executed as a rule. Pack objects are immutable so an evaluation cannot observe a threshold changing midway through a run. The engine receives the pack as an explicit dependency, which makes test fixtures clear and avoids hidden global state.

Pack identity is copied into findings rather than represented only by a current settings value. If a user evaluates under the EU pack, selects the Research Baseline, and later opens an older finding, the finding still displays the EU pack and version. This historical stability is essential for auditability.

The pack contract deliberately does not require every jurisdiction to supply an equivalent

article number. A rule can use a provision, principle, official guidance reference, or non-legal research identifier. The UI renders a general reference label and source link. This avoids encoding the assumption that all legal systems share the GDPR’s structure.

3.19 State, Persistence, and Migration

The privacy context coordinates bridge capability, active pack, loading state, findings, demonstration status, and repository actions. State transitions are explicit: idle, reviewing, completed, and failed or unavailable. The interface disables duplicate actions while a review is running and provides a human-readable result when acquisition returns no evidence.

AsyncStorage is used for app-private persistence. The repository key is versioned. On load, older records are normalised into the current schema where safe. Fields that cannot be reconstructed are marked missing rather than invented. A migration should preserve source identity and original timestamps. If a record cannot satisfy current admission rules, it should be excluded from the active projection and surfaced as a migration issue where appropriate.

Clearing findings removes review records while preserving the application itself. Pack selection may be retained separately because it is a preference, but the UI states what the clear action affects. Android backup is disabled so uninstalling or clearing application data has the expected local-deletion meaning within the platform boundary.

3.20 Security and Abuse Cases

The first abuse case is source forgery: a caller marks synthetic input as native. Enumeration and bridge-controlled construction reduce this risk inside the application, although a modified binary is outside the threat model. The second is numeric abuse: extremely large or non-finite counts cause overflow or misleading totals. Safe-integer validation rejects them. The third is replay amplification. Exact-window replacement provides idempotence. The fourth is overlap amplification, addressed by temporal isolation.

The fifth case is history exhaustion. An attacker or faulty source supplies many distinct windows. Per-key caps and retention limit memory growth. The sixth is identifier pollution, where many malformed package or permission strings create unbounded partitions. Conservative formats and supported permission enumeration reject these values. The seventh is pack confusion, where an unknown identifier is presented under a trusted legal name. Registry-only loading and copied pack metadata prevent silent fallback.

The final abuse case is epistemic rather than computational: a user or stakeholder treats a finding as a legal determination. Interface caveats, neutral statuses, official-source links, and missing-context questions reduce this risk. They cannot eliminate it. Training, documentation, and qualified review remain necessary controls.

3.21 Traceability from Requirement to Evidence

Every requirement is paired with a planned evidence source. Deterministic evaluation is covered by seeded and exact-boundary fixtures. Provenance preservation is covered by type,

repository, and presentation tests. Unsafe inputs are covered by adversarial runtime cases. Pack switching is covered by registry and identity assertions plus Settings rendering. Synthetic separation is covered by simulator-source tests and populated-device screenshots.

Navigation and visual hierarchy are evidenced by UI trees and physical screenshots, while accessibility claims remain limited to source and rendered cues. Local processing and permission minimisation are evidenced by dependency review, merged manifest, and installed-package inspection. Delivery is evidenced by successful release tasks, artefact hashes, package metadata, and installation. The matrix does not list legal compliance or broad user adoption because no test in the current protocol can establish them.

3.22 Evaluation Strategy

Evaluation is deliberately layered. Source-level checks establish type and lint conformance. Deterministic tests establish consistency with the published rule specification and adversarial rejection behaviour. Android builds establish package integration. Manifest and package inspection establish requested-permission facts. ADB interaction and screenshots establish bounded physical rendering and launch behaviour. Each result is reported at its own layer; no result is extrapolated into legal compliance or population effectiveness.

3.23 Design Invariants

The architecture is governed by invariants that should remain true even when screens, acquisition connectors, or legal packs change. First, no accepted finding exists without an admitted evidence source and bounded temporal scope. Second, no legal or research rule exists without a pack identifier and version. Third, unknown, malformed, or unavailable evidence cannot be translated into a reassuring status. Fourth, simulator provenance cannot be upgraded by persistence or presentation. Fifth, historical findings retain the rule identity used at evaluation time. Sixth, local deletion remains available without requiring an account or remote approval.

These invariants are more stable than component names. A future implementation could replace AsyncStorage with Room, React Native with another presentation framework, or bundled packs with signed declarative packages while preserving the same assurance properties. Conversely, a refactor that keeps every current class name but drops source identity or silent-fallback protection would violate the design. Stating invariants therefore makes architectural review less dependent on the incidental code structure of Revision 10.

The invariants also guide test selection. Admission tests challenge the first and third invariants. Pack-switch and persistence tests challenge the second and fifth. Simulator and cross-window tests challenge the fourth. Manifest, backup, and clear-data checks contribute to the sixth. This mapping helps maintainers decide which evidence to renew after a change. It also reveals gaps: authentication of an external producer, for example, is not yet an invariant guaranteed by the prototype and must not be implied by the controlled source label.

An invariant can fail through wording as well as computation. If a finding correctly stores evidence gap but the screen headlines the review as safe, the third invariant has failed at presentation. If an export omits the pack version, the fifth has failed outside the repository.

End-to-end checks are therefore necessary even when unit tests are comprehensive.

Future release reviews should record each invariant as satisfied, failed, or not evidenced, with direct artefact references. A failed invariant blocks promotion. A not-evidenced item may permit a narrower research build, but its associated claim must remain absent. This practice keeps the product’s trust model explicit as functionality grows.

3.24 Decision Records for Future Extension

Future contributors should document material changes as concise decision records. A record should state the problem, evidence, alternatives, selected option, affected invariants, privacy consequences, pack consequences, migration, rollback, and verification required. This is especially important for changes that appear convenient but alter authority, such as adding remote telemetry, inferring jurisdiction automatically, importing an unreviewed rules file, or expanding Android privileges.

A decision record does not replace code review or legal review. It makes assumptions visible before they become distributed across implementation and copy. It also provides a durable explanation when later maintainers ask why the product refuses a seemingly simple shortcut. In a thesis prototype intended for store progression, this continuity matters because deployment work will occur after the original evaluation snapshot.

The first required record for a new acquisition connector should describe deployment authority and observed coverage. The first record for a new regulation pack should identify responsible reviewers and effective sources. The first record for remote services should include a data-flow and retention analysis. The first record for a new user-facing status should specify evidence semantics and comprehension tests. These prompts keep expansion aligned with the central evidence-bounded principle.

4 Implementation

4.1 Application Identity and Delivery Configuration

Revision 9 presents the prototype as *Privacy Lens*, version 1.1.0 (version code 2), under the Android application identifier `com.zhihengzhang.privacylens`. The Android configuration uses minimum SDK 24 and target SDK 36. Release builds generate both an APK for controlled installation and an Android App Bundle for store delivery. The Gradle signing configuration accepts upload-key values through environment variables; when they are absent, local quality-assurance builds use the debug signing fallback and are not represented as production-signed artefacts.

Android backup is disabled. The manifest explicitly removes legacy external-storage permissions. The merged release manifest contains Internet access for official-source links and the operational permissions introduced by WorkManager for wake locks, network state, boot rescheduling, and foreground-service compatibility. It requests no sensitive runtime permission.

4.2 Rule-Pack Engine

The generic rule-pack engine implements validation and threshold evaluation against a supplied regulation definition. The earlier GDPR-named engine remains as a compatibility wrapper that delegates to the EU pack; it no longer owns the rule table. The registry exports only recognised packs and rejects an unregistered identifier.

The EU pack maps supported permission signals to review rationales and official GDPR references. Its caveat states that output is a technical review aid. The Research Baseline pack uses deliberately non-legal terminology. Both conform to the same interface, allowing the Settings screen to change the active pack without changing bridge, repository, dashboard, or navigation code. Pack identity is persisted with every finding, so switching the active selection does not relabel historical evidence.

4.3 Validation and Temporal Isolation

The engine validates package name, permission key, source, count, window, timestamp, and context before evaluation. JavaScript unsafe integers, negative counts, reversed windows, unknown sources, and malformed identifiers fail closed. Simulator provenance is an enumerated source rather than a boolean UI decoration.

The repository uses a versioned local schema and migrates earlier records. Exact replay windows replace prior values. Overlapping aggregate windows cannot be summed; adjacent completed windows remain eligible for a bounded rolling signal. Records are partitioned by package and permission and capped per partition. These constraints make the decision reconstructable while acknowledging that aggregate summaries lack event identifiers.

4.4 Native Android Boundary

Kotlin components implement the React Native bridge, scheduling, permission-audit worker, and controlled violation simulator. The bridge exposes capability status so that the interface can report whether native audit support is present. The product does not claim unrestricted access to other applications' AppOps history. A blank audit is displayed alongside an evidence-reach warning, because stock Android visibility depends on deployment authority and OEM behaviour.

The simulator produces synthetic records for demonstration and regression. Its data source remains explicitly labelled through storage and presentation. This enables a user to inspect a populated workflow without confusing the output with observations collected from the device.

4.5 Information Architecture and Visual Design

The interface was reduced to three primary destinations:

- **Overview** explains the product purpose, initiates an audit, summarises finding classes, and states the evidence-reach boundary;

- **Findings** separates review signals, evidence gaps, and no-concern results, with source and pack labels on each card;
- **Settings** selects a registered rule pack, explains its caveat and official source, runs the synthetic demonstration, and clears local findings.

A custom bottom navigation component replaced the earlier crowded tab structure. The visual system uses deep evergreen, teal, mint, off-white, restrained risk accents, rounded cards, and generous whitespace. The icon combines a shield and lens aperture without text. Status is carried by words and icons as well as colour. Buttons and navigation targets use large touch areas and meaningful accessibility labels, following Android and WCAG design guidance [12, 13].

4.6 Privacy and User-Control Implementation

Findings and active-pack choice use app-private local storage. There is no account, analytics SDK, advertising SDK, remote evidence database, or upload workflow. Settings provides a direct clear-local-findings action. The repository includes a bilingual privacy-policy draft and a store-readiness checklist. The policy distinguishes application behaviour from owner actions still required before public release, including a stable HTTPS policy URL and support contact.

4.7 Codebase Reconciliation

Legacy health-dashboard, project, scoring, experiment, charting, duplicate compliance-service, and unused template components were removed. Dependencies for removed features were also deleted. This cleanup reduces navigation ambiguity and prevents unsupported research screens from appearing to be finished product functions. The resulting codebase has one compliance engine path, one privacy context, one navigation model, and one current user guide.

4.8 Technology Stack and Module Boundaries

The application uses Expo and React Native for the interface, TypeScript for the decision and state layers, AsyncStorage for app-private persistence, and Kotlin for the Android bridge and workers. Expo Router supplies route composition, while React Navigation primitives support the custom bottom navigation. AndroidX WorkManager represents deferrable background work. The release build uses the Android Gradle Plugin, Hermes JavaScript engine, and standard Android packaging tasks.

The stack was selected to separate concerns rather than to maximise the number of frameworks. TypeScript provides shared types for evidence, findings, and packs. Kotlin is limited to capabilities that require Android APIs or native scheduling. The interface does not import Kotlin details; it calls a typed service that reports capability and results. The compliance engine does not import React Native, storage, or Android modules. This makes deterministic tests runnable under Node after TypeScript compilation.

At the source-tree level, regulation definitions reside under the regulation directory, generic decision logic under compliance, application state under context, the bridge adapter under services, and reusable presentation components under a privacy-specific component directory. Theme constants are centralised. This organisation replaces earlier duplicate compliance services and prevents a screen from becoming the authoritative home of a rule.

4.9 Regulation-Pack Implementation

The EU pack is an immutable TypeScript object identified as `EU_GDPR`. It declares the European Union and EEA as its jurisdiction label, cites Regulation (EU) 2016/679 as its version label, and links to the consolidated EUR-Lex source. The pack includes a legal caveat that the automated warning supports accountability review and does not determine infringement.

Three permission families are implemented. Location has a research baseline of 24, microphone a baseline of 8, and contacts a baseline of 4. Each baseline uses a multiplier of 1.5, producing prompt thresholds of 36, 12, and 6 respectively. These values are prototype parameters. They are not extracted from GDPR text, not population estimates, and not represented as legal limits. Their purpose is to exercise an explainable evaluation path and provide deterministic boundaries for testing.

The EU pack supplies a rationale and reference for every permission family. Location directs review toward necessity, proportionality, and documented lawful basis under Articles 5 and 6. Microphone review includes the possibility that special-category material may require Article 9 analysis. Contacts review emphasises purpose limitation, minimisation, and lawful basis. The reference is intentionally a prompt rather than an assertion that the cited provision applies to every record.

Missing-evidence logic is also pack-owned. The EU pack asks for specified purpose, Article 6 lawful basis, controller identity, retention period, and, when special-category processing is marked, an Article 9 condition. If processing relies on consent, consent has been withdrawn, and a positive access count remains, the pack returns its most urgent review status. If required context is missing, it returns insufficient evidence. Otherwise a threshold signal produces review required, while the absence of a signal produces no technical concern for the admitted check.

The Research Baseline pack uses higher prototype baselines of 36, 12, and 6 before applying the same multiplier. It uses region-neutral principles and does not require an Article 6 or Article 9 field. Its caveat explicitly states that it is not law or legal advice. The difference demonstrates that the engine can consume pack-owned rules and missing-evidence logic without embedding GDPR branches in generic code.

4.10 Runtime Validation Pipeline

The safe evaluation entry point accepts an unknown value rather than trusting its compile-time annotation. The first check requires a non-array object. The package identifier must be a string between one and 255 characters using a conservative character set. Permission type must belong to the owned set of location, microphone, and contacts. Source, if present, must be

simulator, native bridge, or imported. This sequence prevents later operations from assuming properties that have not been established.

The access count must be a non-negative safe integer. The window start and end must also be safe integers, the start must precede the end, duration cannot exceed one day, and the end cannot be implausibly far in the future. When event timestamps are supplied, their array length must equal the access count and every timestamp must lie within the audit window. These conditions ensure that peak-rate calculations do not operate on inconsistent evidence.

Processing context is validated field by field. Purpose, controller identity, and Article 9 condition are optional strings. Consent-withdrawn, special-category, and user-initiated values are optional Booleans. Lawful basis must belong to the Article 6 enumeration used by the prototype. Retention days must be a non-negative safe integer. Values are rejected rather than coerced because coercion can change meaning at a trust boundary.

Every rejection returns an explicit code and message with a rejection timestamp. The throwing evaluation method is retained for callers that require an exception, but the application can use the safe result to display a controlled evidence gap. This dual interface allows strict internal use without forcing the UI to recover from an untyped exception.

4.11 Signal Computation

After admission, the engine loads the rule for the active pack and computes the prompt threshold

$$T_p = \max(1, \lceil B_p M_p \rceil), \quad (2)$$

where B_p is the pack baseline and M_p is the deviation multiplier. Daily excess is $\max(0, x - T_p)$, where x is the admitted count. A daily-total signal is emitted only when excess is positive, so equality with the threshold remains below the signal boundary.

When event timestamps are present, the engine sorts them and evaluates a sliding sixty-second interval. Two indices move monotonically through the sorted array, so peak calculation is linear after sorting. Permission-specific burst limits are 12 for location, 6 for microphone, and 4 for contacts. A burst signal is emitted when the observed peak is strictly greater than the configured limit.

The history key combines package and permission. Previous entries outside the one-day watermark are discarded. An entry with the same start and end is treated as replay and replaced. Only entries ending no later than the current start contribute to the completed rolling total. Overlapping entries stay in bounded history but do not contribute to that total. A cross-window signal is disabled for simulator evidence and requires at least one compatible completed window.

Risk derives from the greatest normalised excess across daily, burst, and rolling dimensions. Ratios below the first threshold remain low; larger ratios map through medium, high, and critical bands. Risk controls presentation and notification priority, while the pack separately controls the compliance-review status. Keeping these concepts distinct prevents a large

technical deviation from bypassing missing legal context.

4.12 Finding Construction and Communication

The finding identifier combines package and permission so the repository can update a stable subject. Evidence fields contain daily count, peak calls per minute, rolling count, and source. Decision fields contain threshold, excess count, active signals, risk, detection time, regulation identifier and name, legal or research reference, and rationale. Compliance fields contain status, applicable principles, missing evidence, and caveat.

Communication is derived after the finding exists. The title identifies the permission review. The summary translates status and signal identifiers into human-readable language. The recommended action first requests missing evidence; an urgent result suggests pausing processing where appropriate and obtaining human review; other results recommend examining purpose, necessity, and proportionality. Notification priority is urgent only for the strongest status or critical risk, silent for no technical concern, and standard otherwise.

This construction ensures that UI wording is based on stored decision state. A card is not permitted to infer a legal label merely from colour or excess count. It receives the pack name, source, caveat, and missing context that were produced with the finding.

4.13 Repository Behaviour

The repository uses a versioned AsyncStorage key. Listing parses the stored array, replacement writes at most the bounded presentation set, clearing removes the key, and saving updates an existing package-permission finding or appends a new one. The latest application context also normalises records created by earlier schemas so that pack identity and evidence fields remain available where they can be reconstructed safely.

Notification state is based on change. A new finding can notify; a reversal of active state can notify; an increase in risk rank can notify. A byte-for-byte identical record is not treated as changed. This policy prepares the product for attention-aware notification without claiming that a longitudinal notification campaign has been conducted.

The decision engine's internal temporal history and the persistent finding repository serve different purposes. Engine history supports signal computation within a running context. Repository findings support user review across sessions. Conflating them would either store unnecessary event aggregates indefinitely or make visible findings disappear whenever the process restarts.

4.14 Android Bridge and Scheduling

The Kotlin module reports whether the native privacy inspector is available and exposes bounded audit and demonstration operations to React Native. WorkManager workers are registered for periodic or one-time tasks according to Android's scheduling model. WorkManager is deferrable: the operating system may delay work according to battery, idle state, quotas, and vendor policy. The implementation therefore does not describe a registered interval as

proof of exact execution time.

The audit worker reads only evidence available under the application’s deployment authority. On the tested ColorOS device, that authority did not expose unrestricted third-party history. The bridge returns capability and available summaries rather than fabricating other-package events. The simulator worker follows a controlled path and marks every generated record as simulator source.

Native package declarations were moved from the anonymous template namespace to the stable Privacy Lens namespace. Activity, application, bridge, package, and worker source files use the same identifier as Gradle and the manifest. This reconciliation matters for release upgrades, signing continuity, deep links, provider authorities, and store identity.

4.15 Interface Implementation

The Overview screen has four layers. A brand header establishes identity and provides a direct Settings entry. A high-contrast hero panel explains the evidence-before-conclusions proposition and presents the primary review action. A current-review section summarises needs-review, evidence-gap, and no-concern counts. An evidence-reach card explains whether the native bridge is available and why an empty result is not proof of no access.

The Findings screen is designed for both empty and populated states. The empty state explains what will appear and directs the user back to a review or demonstration. The populated state renders reusable finding cards. Each card shows status, permission, package, source, regulation pack, detected signals, rationale, and missing context. Source and regulation are placed near the result because they alter how the result should be interpreted.

The Settings screen separates preference, explanation, demonstration, and deletion. Pack selection is restricted to registry entries. Selecting a pack displays its jurisdiction, version, description, caveat, and official source. The controlled demonstration is visibly separated from device review. Clearing findings uses a destructive visual treatment and explanatory copy.

The custom bottom navigation uses three equally weighted targets with icon, label, accessibility role, selected state, and sufficient touch area. It replaced the platform tab implementation after physical coordinate and hierarchy evidence revealed ambiguity in the previous navigation setup. Navigation semantics no longer depend on obsolete routes.

4.16 Brand and Accessibility Implementation

The visual language uses evergreen for trust and stability, teal for actions, mint for supportive surfaces, warm off-white for the application background, and restrained red and blue accents for review and evidence states. Rounded panels group related information without relying on heavy separators. Typography uses strong size contrast between product proposition, section heading, count, label, and explanatory copy.

The application icon combines a shield with a lens aperture. The shield suggests protection while the aperture suggests inspection; neither implies surveillance of a person. Adaptive foreground and monochrome assets support launcher contexts. Splash imagery, application

icon, colours, and product name are generated consistently across Android densities.

Accessibility labels are supplied for navigation and important actions. Icons do not carry status alone. Buttons use text and large target areas. Cards maintain text contrast against their surfaces. These implementation facts support an accessibility-oriented design claim, while screen-reader traversal, large-font reflow, and assistive-technology user testing remain future acceptance work.

4.17 Build, Permission, and Signing Configuration

The Android build sets minimum SDK 24, compile and target SDK 36, version code 2, and version name 1.1.0. Release tasks produce an APK for controlled device installation and an AAB for Play delivery. The build reads upload-store path, passwords, and alias from environment variables. If they are absent, a local QA fallback signs with the debug configuration. The build can therefore be tested without placing secrets in source, while its output remains clearly distinguished from a production upload artefact.

The source manifest requests Internet access and explicitly removes inherited legacy external-storage permissions using manifest-merger directives. The final merged manifest also contains operational declarations contributed by WorkManager and AndroidX. Android backup and data extraction are disabled for the application. Inspection of the installed release package confirms that no sensitive runtime permission is requested.

Package reduction was part of implementation. Dependencies used only by removed charts, health views, haptics, image helpers, secure storage, web browser wrappers, gestures, SVG rendering, and network-status features were removed when no longer needed. Generated build and cache directories remain ignored. This reduces attack surface and maintenance burden, although transitive dependencies are still reviewed through the lock file and final manifest.

4.18 Implementation Summary

The implemented system realises the Chapter 3 contracts: inputs fail closed, evidence provenance survives the pipeline, packs are replaceable through a registry, the interface avoids legal verdict language, and Android delivery artefacts use a stable identity. Chapter 5 evaluates these properties at source, rule, build, manifest, and physical-device layers.

5 Evaluation and Results

5.1 Evidence Model

The evaluation separates five evidence layers: source conformance, deterministic rule behaviour, adversarial boundary behaviour, Android package integration, and physical-device usability evidence. This separation prevents a passed unit test from being reported as device reliability or legal validity. The final acceptance record is stored at `testing-report/release-candidate-2026-08-09.md`; screenshots and UI hierarchies are retained in the adjacent dated folder.

5.2 Source and Rule Verification

The complete application passed `tsc -noEmit` and ESLint. The compliance sources were compiled before execution, preventing stale generated JavaScript from being treated as current evidence.

A fixed-seed logical campaign generated 1,000 cases. Eight hundred were expected review signals and 200 were below-threshold controls. The observed confusion matrix was

$$(TP, TN, FP, FN) = (800, 200, 0, 0), \quad (3)$$

which gives precision and recall of 1.0000 against the specified oracle. The oracle and engine implement the same published threshold model; this is a conformance and regression result, not an independent estimate of real-world infringement detection. Zero observed errors do not establish a zero population error rate.

Extended suites passed for unsafe types and integers, malformed context, unknown source, permission-key mutation, temporal overlap, adjacency, replay, retention, simulator isolation, dashboard presentation, missing-context disclosure, and regulation-pack identity. The tests confirm the enumerated boundaries. They cannot establish correctness for telemetry the application never receives or for legal interpretation outside the fixtures.

5.3 Release Build and Manifest Verification

Gradle completed `assembleRelease` and `bundleRelease` with target SDK 36. Table 2 records the generated artefacts.

Table 2: Revision 9 Android release artefacts

Artefact	Bytes	SHA-256
<code>app-release.apk</code>	62,762,819	4A5BCED7FBF6A7A5A0E337BFAADB003A3C7 FE89FDAFBF9BADCB6BC4CBFF51B5
<code>app-release.aab</code>	31,700,025	E4090FEEF4EFBD0B697F863D80D28F289C3 554BA89394A1435EACA2C226ACF78

The merged release manifest contained declarations for Internet access, wake lock, network state, boot rescheduling, foreground service, and an app-scoped signature permission. External-storage read and write permissions were absent. Inspection of the installed package confirmed the same requested-permission set and no runtime permissions. Android backup was disabled in the application declaration.

5.4 Physical-Device Evaluation

The release APK was installed by ADB on one OPPO PERM00 ARM64 device. The installed package reported `com.zhihengzhang.privacylens`, versionName 1.1.0, versionCode 2, minSdk 24, and targetSdk 36. Launcher cold start succeeded. A filtered scan of re-

cent `AndroidRuntime` and `ReactNativeJS` error channels returned no fatal match during the acceptance run.

The three primary destinations rendered at the device’s captured application resolution. Coordinate-driven exploration initially appeared to show a navigation-target problem because the physical display and captured application coordinate spaces differed. A bounded coordinate sweep and UI-tree inspection identified the offset; subsequent navigation and custom bottom-bar interaction succeeded. This calibration episode is retained because it demonstrates why raw ADB coordinates must not be interpreted without the target’s current window geometry.

The controlled demonstration completed a 50-record synthetic workflow. The populated overview displayed four review items, zero evidence gaps, two no-concern items, and descriptive precision and recall of 100% for the simulator’s known labels. The Findings screen marked the source as synthetic and displayed the EU GDPR pack. These values validate the demonstration path only; they are not device-observation accuracy.

5.5 Usability and Trust Assessment

The product was assessed against three release-candidate questions.

Table 3: Bounded product assessment

Criterion	Supporting evidence	Boundary
Visually coherent	consistent brand, hierarchy, spacing, cards, iconography, and three-destination navigation on the tested device	one device; no participant preference study
Trustworthy	local-first disclosure, source and pack labels, explicit missing evidence, no legal-verdict language	not a legal audit or security certification
Willing to use	clear first action, concise summaries, optional details, demonstration, and direct data clearing	evaluator judgement; no longitudinal adoption measure

The evaluator judged the candidate to meet the project’s “beautiful, trustworthy, and willing to use” gate for an initial submission candidate. This is a structured expert judgement supported by rendered evidence, not a statistically generalisable user study.

5.6 Evaluation Questions and Acceptance Logic

The evaluation was organised around six questions. First, does the source compile and satisfy static rules? Second, does the decision engine agree with its specified thresholds and statuses? Third, does it reject malformed or adversarial evidence safely? Fourth, do regulation packs remain independent and traceable? Fifth, can the Android release artefacts be assembled,

inspected, installed, and launched? Sixth, does the rendered product communicate the expected hierarchy and evidence boundaries on the available physical device?

Each question has a distinct acceptance condition. Source acceptance requires a clean TypeScript application check and ESLint run. Rule acceptance requires exact expected confusion-matrix values for the fixed corpus and passing boundary assertions. Runtime-boundary acceptance requires every malformed case to return the specified rejection code without an uncaught exception. Pack acceptance requires registry allowlisting, different thresholds where defined, retained pack identity, and a non-legal caveat for the Research Baseline.

Android build acceptance requires both release tasks to complete and the artefacts to exist with recorded hashes. Permission acceptance requires agreement between the merged manifest and installed package, with legacy external-storage permissions absent. Physical acceptance requires successful streamed installation, launcher start, stable primary navigation, captured final states, and no matching fatal error in the bounded log scan. Visual acceptance requires inspection of actual screenshots rather than source code alone.

The protocol deliberately does not set an acceptance condition for GDPR compliance, legal accuracy, population detection accuracy, long-run scheduler reliability, battery consumption, multi-device compatibility, or store approval. Those outcomes require evidence not collected in this campaign.

5.7 Reproducibility Controls

The compliance sources are compiled immediately before the generated JavaScript runners execute. This control was introduced because a failed TypeScript command can otherwise leave older generated output in place, producing a misleading green test. The acceptance sequence checks the compiler exit code before invoking Node. Generated compliance output is ignored by Git so it cannot become an accidental source of truth.

The main logical corpus uses a fixed linear-congruential seed. A fixed seed makes a failure reproducible while still exploring varied generated values. The 1,000 rounds preserve an 80/20 design: four of every five cases are expected review signals and one is a control. A fresh engine is created for each main-corpus case so temporal state from one generated subject cannot leak into another.

Boundary cases are not left to random chance. For every supported permission, counts at threshold minus two, minus one, equal, plus one, and plus two are exercised. The assertion encodes the strict boundary: only values above the prompt threshold become active under the daily-total rule. Burst tests similarly check exactly at and exactly above each permission-specific limit.

Time is fixed in extended tests. Windows and timestamp arrays are created relative to a constant epoch value rather than the wall clock. This prevents a test from becoming invalid while it runs and makes future failures comparable. Production validation still compares implausible future times against the current clock, but fixtures choose values far enough in the past to avoid accidental rejection.

5.8 Deterministic Corpus Interpretation

The confusion matrix compares the engine’s active-state decision with the generator’s specified expected label:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (4)$$

The resulting precision and recall of 1.0000 mean that the implementation agreed with the oracle for all generated cases. This supports a regression claim: later refactoring did not alter the expected threshold semantics for the corpus. It also supports a fixture-quality claim: the corpus contains both positive and negative cases and the metric implementation returns the expected counts.

It does not show that a threshold crossing identifies unlawful processing. The expected label is generated from the same documented research model that the engine is meant to implement. Although the oracle is expressed independently of the evaluated result, both are derived from one specification. A defect in that specification could be reproduced consistently. There is no independently labelled sample of real applications or legal cases.

The tests also verify metric edge conditions. Empty samples, all-negative samples, and samples with missed positives must produce finite precision and recall rather than not-a-number or infinity. This matters because a dashboard can otherwise display an apparently precise but meaningless metric when a denominator is zero.

5.9 Adversarial Input Results

The initial security cases reject unsupported permission values, not-a-number counts, negative counts, reversed windows, and prototype-like permission keys. Extended cases add unsafe integers and positive infinity. Every such count returns the invalid-count code. Timestamp evidence is rejected when array length differs from count, when an element falls before the start, when it falls after the end, or when it is fractional.

Processing context is attacked through type confusion. Numeric purpose, array controller identity, object Article 9 condition, string consent-withdrawn, numeric special-category, and string user-initiated values all return invalid context. An unknown lawful basis and negative retention period are also rejected. These cases are significant because JavaScript truthiness or an early string operation could otherwise convert malformed evidence into a plausible status or throw outside the controlled error path.

Source validation rejects a forged source. This prevents a caller from using an arbitrary label that resembles an authoritative acquisition path. Package and permission history are isolated: a high-count record for one package cannot make another package active. Histories are also isolated by permission family.

The assertions demonstrate the implemented rejection surface, not complete security. They do not cover a compromised native module, a modified application binary, corrupted AsyncStorage beyond the migration parser, malicious operating system, or every possible Unicode identifier. Security conclusions remain bounded to the enumerated input contract.

5.10 Temporal and Replay Results

Burst tests create timestamp arrays exactly at and just above the location, microphone, and contacts limits. At-limit arrays produce no burst signal; one additional event produces the signal. A separate imported-evidence example concentrates 20 location events within one minute and is detected.

Cross-window tests use two below-threshold location summaries. The first remains inactive. When the second begins after the first has completed, their rolling total exceeds the threshold and the engine emits a cross-window signal. The same construction is repeated with adjacent windows to establish that equality between a prior end and current start is admissible.

An overlapping construction uses one interval that intersects the next. The second finding's rolling count remains its own count and no cross-window signal appears. This is the expected conservative behaviour because aggregate evidence cannot identify duplicate events in the overlap. An exact replay is then evaluated twice before a compatible next interval. Rolling total confirms that the repeated window contributed once, not twice.

Simulator histories are excluded from cross-window signalling. The demonstration may still produce a daily or burst signal for a record, but it cannot present accumulated simulator activity as imported temporal evidence. This is a provenance safeguard, not a statistical assumption.

The temporal tests operate in process memory and do not demonstrate persistence of aggregate history across process death. They establish the logic of the current engine instance. A production design requiring durable cross-window evidence would need a versioned evidence-history repository and migration tests separate from the visible finding repository.

5.11 Accountability and Presentation Results

A low-count contacts record without processing context returns insufficient evidence rather than no concern. Its communication priority is standard because the absence of legal context warrants review. A low-count location record with purpose, contract basis, controller identity, one-day retention, and user-initiated context returns no technical concern and a silent priority.

A microphone record marked as consent-based, special-category, and accessed after consent withdrawal returns the urgent review status. The fixture does not prove a real GDPR infringement; it confirms that the pack's stated escalation branch is implemented. A special-category microphone record without an Article 9 condition lists that condition among missing evidence. An invented lawful basis and invalid retention value are rejected at admission.

Dashboard projection preserves three different states from the accountability fixtures. Summary totals show one action item, one evidence gap, and one no-concern result. Filters surface the expected finding. Every presentation state has both a textual label and marker, ensuring that source-level semantics do not depend on colour alone.

5.12 Regulation-Pack Results

The registry exposes at least two packs and recognises only the two expected identifiers. A forged identifier is rejected. The same location audit with a count of 40 is evaluated under both packs. The resulting findings retain different regulation identifiers and different thresholds because each pack supplies its own baseline.

This test is stronger than checking that a dropdown changes text. It passes the same admitted evidence through two independently instantiated engines and inspects the resulting finding objects. The Research Baseline result also contains a caveat stating that it is not law. The test does not validate either pack’s legal interpretation; it validates decoupling, identity, and disclosure.

Future pack validation should add a contract suite automatically applied to every registry entry. It should require a unique identifier, version, official-source policy, complete supported-rule set, positive finite baselines and multipliers, non-empty rationale and caveat, deterministic missing-evidence logic, and fixtures reviewed by the responsible domain expert.

5.13 Build Reproduction

The release build initially encountered Windows path and dependency-layout problems. A hoisted local dependency layout allowed native CMake paths to remain within workable bounds. React Native’s new architecture was disabled for the release candidate because enabling it in this Windows checkout reproduced path-related native build failures. This is a build-environment decision, not evidence that the legacy architecture is preferable for future releases.

Gradle reported deprecated features that will require attention before Gradle 9 compatibility. It also reported an SDK XML version warning caused by differences between installed Android Studio and command-line components. Neither warning prevented the recorded release build, but both are retained as maintenance debt rather than suppressed.

The release task generated bundles for multiple Android ABIs. The APK was installed on the ARM64 physical device. The AAB was not uploaded to Play Console, processed by Play signing, or delivered through an internal track. Its success establishes local bundle generation only.

Hashes were recorded immediately after the final permission-manifest rebuild. Any source or build-tool change requires rebuilding and computing new hashes. The files are generated artefacts and are not treated as immutable simply because their names remain the same.

5.14 Manifest and Installed-Package Reconciliation

Source-manifest inspection alone was insufficient because dependencies contribute declarations through manifest merging. The initial merged manifest exposed legacy external-storage permissions inherited from transitive Expo components. Revision 10 added explicit merger removals for read and write external storage, rebuilt both artefacts, and inspected the merged result again.

The final merged manifest contained Internet, wake-lock, network-state, boot-completed, foreground-service, and an application-scoped dynamic-receiver signature permission. These operational declarations are consistent with linking and WorkManager components. The installed-package dump reported the same requested set and no runtime permission grants. This two-level check supports the documentation statement that no sensitive runtime permission is requested.

The package dump also confirmed application identity, version name, version code, minimum SDK, target SDK, primary ABI, install state, and signing version. The release used the QA fallback signing identity because production environment variables were absent. A successful package inspection is therefore not proof of final upload-key configuration.

5.15 Physical Interaction Protocol

The device was identified by ADB before installation. The final release APK was streamed with replacement enabled, the package was force-stopped, and the launcher intent was injected. Package metadata was read after installation so that a stale or differently named package could not be mistaken for the new build.

Interaction evidence included screenshots and UI hierarchy dumps for overview, findings, and settings. A controlled demonstration populated the dashboard and finding list. The final overview was captured again after the permission-manifest rebuild and installation. Runtime review used filtered AndroidRuntime and ReactNativeJS error channels; no matching fatal output appeared in the bounded recent log window.

An important measurement issue arose from coordinate spaces. The physical display reported 1080 by 2400 pixels, while the captured application window and hierarchy used 1080 by 2244. Initial taps derived from the full display therefore appeared to miss navigation targets. A bounded coordinate sweep and hierarchy inspection established the offset, after which navigation succeeded. The event is reported because excluding it would conceal a real threat to ADB-based UI-test validity.

Future automation should select accessibility nodes or stable resource identifiers instead of relying on raw coordinates. It should record display size, application bounds, system-bar insets, orientation, density, and font scale for every run. Screenshots should be associated with the installed package hash and build hash so that visual evidence cannot drift from the tested binary.

5.16 Visual Review Method

Visual assessment considered hierarchy, identity, first-action clarity, state distinction, provenance visibility, limitation placement, empty states, populated states, navigation consistency, destructive-action treatment, and overall coherence. The assessment did not score aesthetic preference numerically because no validated participant scale was administered.

The overview succeeded by making the product proposition and review action dominant while keeping the evidence-reach warning in the first viewport. Summary cards used count, icon, label, and colour. Findings showed synthetic source and active pack near the decision.

Settings gave the pack caveat and controlled demonstration their own sections. The custom navigation remained legible against the off-white background and used a clear selected state.

The result satisfies a project-defined candidate gate rather than a universal usability standard. One evaluator and one physical device cannot establish willingness to adopt across a population. The strongest supported statement is that the rendered candidate is coherent, presents its boundaries, and is suitable for internal-track and participant evaluation.

5.17 Result Boundary

The evidence supports the claim that the specified code compiles, the rule engine conforms to its fixtures, the Android artefacts build, the final package installs and launches, the main interface renders coherently, and provenance remains visible in the tested workflows. It does not support claims about 24-hour execution, battery cost, memory leaks, unrestricted third-party observation, multiple OEMs, legal compliance, accessibility conformance, production signing, or store acceptance.

5.18 Evidence Triangulation

No single method in the campaign was designed to carry the complete acceptance claim. Source inspection is good at confirming constants, branches, labels, and data flow, but it cannot establish which code entered an APK or how a screen rendered. Automated tests execute specified behaviours repeatedly, but their assertions can share assumptions with the implementation. Package inspection observes the assembled artefact, but it cannot prove the meaning of every runtime path. Device interaction observes a concrete installation, but the sample is narrow. The evaluation therefore uses convergence among methods while preserving the scope of each.

The deterministic tests and source review converge on threshold semantics. Source review identifies the pack-provided baseline, multiplier, daily threshold, burst limit, and source restrictions. Boundary assertions then test values immediately around those decision points. The fixed corpus tests varied inputs at scale. Agreement across these methods reduces the chance that the reported rule differs accidentally from the implemented one. It still does not establish that the rule is a valid legal classifier.

Runtime-boundary tests and presentation tests converge on uncertainty handling. Malformed evidence is rejected at admission, low-context evidence becomes an evidence gap, and presentation projection retains that state. This end-to-end check matters because a correct engine could be undermined if the UI translated an unknown state into reassuring copy. The tests show that selected fixtures preserve meaning through the current projection; screenshot review confirms the principal labels are visible in the rendered flow.

Build logs, output existence, checksums, manifest inspection, installed-package metadata, and physical launch converge on artefact identity. Each closes a different substitution risk. A successful Gradle message without an output file would be insufficient. A local file without a checksum could be confused after rebuilding. Source manifest facts could differ after dependency merging. An installed package could be a stale version. Reading these signals together supports

the conclusion that the evaluated handset ran the recorded release candidate.

Visual review and UI-hierarchy inspection also complement each other. A screenshot shows layout, colour, clipping, and hierarchy as a person sees them. A hierarchy dump exposes text, bounds, and accessibility-node structure that may not be obvious visually. Neither replaces screen-reader interaction, large-font testing, or participant comprehension. Their combined use supports a bounded expert assessment and identifies the next accessibility evidence required.

Triangulation is not vote counting. If one authoritative check contradicts another, the discrepancy must be resolved rather than outvoted. For example, an installed-package dump showing a sensitive permission would invalidate a source-only conclusion even if several documents said otherwise. The evaluation gives greater weight to evidence closest to the released artefact for package claims and to independent human or legal review for substantive claims.

5.19 Oracle Independence and Correlated Defect Risk

The main corpus labels are derived from the same published threshold model the implementation is intended to realise. This is appropriate for conformance testing: it asks whether the program implements the specification. It is insufficient for empirical validation because a mistaken specification can yield perfect agreement. The measured 1.0000 precision and recall are therefore implementation-oracle agreement values, not estimates of performance on real legal cases.

Several design choices reduce accidental coupling. The generator records expected polarity directly from its construction, a fresh engine limits cross-case state, boundary cases are written explicitly, and extended tests target properties beyond the main daily-total rule. Nevertheless, developer knowledge shaped both sides. The campaign did not use a separately developed executable oracle, blinded labels, or an external benchmark.

Mutation testing would provide one useful next check. Deliberately changing a comparison from greater-than to greater-than-or-equal, weakening source validation, removing replay replacement, or allowing overlapping history should cause specific tests to fail. Surviving mutations would identify assertions that execute code without distinguishing correct behaviour. Property-based generation could then explore many valid and invalid shapes while preserving stated invariants.

An independent reference implementation could further reduce correlated defects. It should be written from the regulation-pack contract by a reviewer who has not read the production engine, ideally in a different language or style. Both implementations could evaluate a versioned interchange corpus. Disagreement would trigger case analysis rather than majority voting. This process would validate formal semantics, though it would still not validate legal relevance.

For legal or ecological validation, the oracle must change. Controlled event traces can provide ground truth about whether acquisition observed an action. Independent privacy reviewers can label whether a scenario warrants investigation, using a documented protocol and recording disagreement. Neither source should be described as perfect legal truth. The evaluation question, sampling frame, and annotation uncertainty must be reported alongside

performance metrics.

This distinction explains why the thesis retains the term deterministic corpus rather than benchmark. A benchmark normally suggests comparison against an external reference population. The current corpus is a rigorous regression artefact that protects rule semantics as the code changes. Its value is high, but different from an independently labelled real-world set.

5.20 Metric Interpretation and Error Analysis

Precision and recall were included because they expose false-positive and false-negative cells more clearly than accuracy in an imbalanced corpus. The designed 80/20 mixture would allow a trivial always-positive strategy to appear 80 per cent accurate. The full confusion matrix prevents that reading: all 200 controls were negative and all 800 constructed signals were positive under the specified oracle.

The exact totals also make the experiment reproducible. The campaign did not sample a changing external population; it evaluated a fixed generated set. Confidence intervals around the observed proportions would describe repeated sampling from an assumed generator, not uncertainty about the fixed run. More importantly, they would not address specification validity. For this reason the report presents exact agreement and claim boundaries instead of using a narrow interval to imply real-world certainty.

Error analysis remains meaningful even when the main matrix has no disagreements. Adversarial suites reveal categories the primary generator does not represent: unsafe numeric values, prototype-derived identifiers, malformed context, source forgery, temporal overlap, replay, and history isolation. These tests show that corpus size alone is not coverage. A small, carefully targeted case can provide more security evidence than hundreds of additional values drawn from the same safe range.

Future field studies should report more than aggregate precision and recall. Results should be stratified by permission, application category, source authority, foreground status, OEM, Android version, and window resolution. Reviewers should inspect false positives and negatives for common causes. Missing acquisition records should not be treated as model negatives. Cases excluded because evidence is insufficient should be counted and explained rather than removed from the denominator without notice.

Calibration should be evaluated separately from discrimination. A status hierarchy may rank more concerning cases correctly while its displayed wording still conveys excessive certainty. Participant studies can test whether confidence changes appropriately between authoritative, synthetic, and incomplete evidence. This human calibration outcome is central to the product's trust claim and cannot be inferred from the engine matrix.

Operational metrics also need denominators. A launch success rate requires a defined number of clean starts and devices. A crash-free percentage requires a meaningful observation period and collection method. A jank percentage depends on a representative interaction trace and enough rendered frames. The current evaluation reports bounded events directly rather than producing population-style percentages from a short sample.

5.21 Store-Candidate Evidence Review

The project-defined store-candidate gate was assessed across product identity, core journey, disclosure, packaging, privacy posture, accessibility foundations, and operational ownership. Product identity was supported by a consistent name, icon, visual language, and package identifier. Core journey was supported by first-launch overview, findings navigation, detailed rationale, pack selection, demonstration control, and local-data deletion. Disclosure was supported by visible source labels, pack identity, evidence limits, and non-verdict language.

Packaging evidence included release APK and AAB generation, target SDK 36, ABI outputs, final checksums, manifest reconciliation, and installation of the APK. Privacy posture included no sensitive runtime permission in the final package, disabled backup, local-first storage, and a draft privacy policy consistent with the tested build. Accessibility foundations included textual state labels, non-colour cues, labelled controls, target sizing intent, and a simple navigation structure.

The gate was not interpreted as proof that every store prerequisite is complete. Production upload signing was absent. The privacy policy required owner-controlled HTTPS publication. Console declarations, listing assets, support details, content rating, internal-track delivery, and store review remained external. These gaps do not negate the engineering candidate; they determine why it must not be described as a publicly released production version.

The visual adjectives in the gate were operationalised cautiously. “Beautiful” meant coherent hierarchy, spacing, typography, colour restraint, branded identity, and absence of visible broken states in the inspected flows. “Trustworthy” meant that evidence provenance and limitations were prominent, destructive controls were explicit, and the product did not present a legal badge. “Willing to use” meant that an evaluator could understand the purpose, reach findings, inspect reasons, switch the review framework, and control data without encountering a blocking usability defect.

These definitions create a repeatable expert gate, but they do not convert subjective judgement into population evidence. A single evaluator can miss cultural preferences, accessibility barriers, unfamiliar terminology, or long-term fatigue. The appropriate next step is internal-track and participant evaluation using the actual distributed candidate. The expert gate answers whether that investment is justified, not whether the market has accepted the product.

The candidate judgement also depends on the declared product category. A local research and accountability aid can be useful with bounded imported or simulated evidence. A product marketed as universal device surveillance would fail because the acquisition authority is not established. Store copy must preserve the former positioning. Changes to marketing are therefore capable of invalidating readiness even if the binary remains identical.

5.22 Evaluation Repeatability and Change Control

The evidence applies to a specific source revision and generated artefacts. Repeating only the final UI flow after changing engine code would not renew rule evidence. Rebuilding only the APK after changing privacy copy would not renew screenshot evidence. The project therefore

treats acceptance as a dependency graph: a change invalidates every downstream artefact that incorporates or presents it.

Engine or pack changes require TypeScript compilation, logical suites, boundary suites, presentation projections, release rebuilds, hashes, and targeted device review. Android manifest or Gradle changes require merged-manifest inspection, package dump reconciliation, rebuilds, installation, and launch. UI changes require lint, package build, hierarchy and screenshot capture, accessibility checks, and renewed expert review. Policy or store-description changes require reconciliation with package behaviour and data flows.

Reproducible scripts reduce omission but do not remove the need to inspect their outputs. A command may exit successfully while testing stale generated code, selecting the wrong device, or reading a prior file. The campaign therefore compiles generated runners immediately before execution, checks package identity after installation, and associates evidence with checksums. Future automation should emit a machine-readable manifest linking commit, tool versions, commands, artefact hashes, device facts, and screenshots.

Environment details also matter. Windows path length and native dependency layout affected release compilation. The chosen workaround and architecture flag belong in the build record because a different environment may produce different outputs. Tool warnings about future Gradle compatibility remain maintenance observations even when the current build passes.

A release tag should be immutable, while packs and policies use explicit version identifiers. If a later investigation finds an error, the project should publish a successor and a correction note rather than rewriting the historical evidence. This thesis follows the same principle by creating Revision 10 chapter files instead of replacing Revision 9. Preservation makes it possible to see which claims changed and why.

Repeatability finally includes honest failure recording. Network, dependency, device-coordinate, and build-path failures can be part of the evidence because they expose operational assumptions. Removing them from the narrative would make future reproduction harder. A trustworthy evaluation explains both the final pass and the conditions needed to obtain it.

6 Discussion and Limitations

6.1 What the Prototype Establishes

Privacy Lens demonstrates that a permission-review product can be useful without claiming complete visibility or automated legality. The core value is not the threshold alone; it is the evidence contract surrounding the threshold. Source, window, pack, rule reference, and missing context make the decision traceable. Fail-closed validation prevents malformed evidence from becoming a reassuring result. The interface then communicates the distinction between an observed signal, a synthetic example, and an evidence gap.

The replacement of a GDPR-specific engine with regulation packs also improves scientific clarity. It reveals which statements belong to generic software and which depend on a jurisdictional interpretation. This supports future extension while making clear that interface

compatibility is not legal equivalence.

6.2 Construct and Internal Validity

The deterministic confusion matrix measures agreement with a defined threshold oracle. It does not measure whether that threshold identifies unlawful processing. Because test fixtures and implementation share the same conceptual specification, correlated defects remain possible. Boundary suites reduce but do not eliminate this risk. Independent implementation review, mutation testing, and event-level reference traces would strengthen internal validity.

The simulator is intentionally useful for reproducible demonstration, but it can inflate perceived confidence if its labels are mistaken for field ground truth. Revision 9 mitigates this by preserving synthetic provenance through the repository and interface. The remaining risk is user over-reliance on any precise metric; future builds should consider hiding precision and recall outside an explicitly labelled research panel.

6.3 External and Ecological Validity

Physical evaluation used one OPPO PERM00 device and a short acceptance window. OEM process management, Android versions, accessibility settings, screen sizes, font scaling, languages, network states, and long-run scheduling can change behaviour. No claim is made for 24-hour WorkManager reliability, battery impact, memory leak freedom, or multi-device stability. A release-candidate status at this layer means the tested artefact is coherent enough for controlled store submission work, not that broad compatibility is established.

The ADB coordinate calibration problem also illustrates a methodological limitation: automated interaction evidence depends on the relationship between physical display coordinates, application-window bounds, navigation bars, and screenshots. Future UI automation should anchor actions to accessibility nodes or resource identifiers and record window geometry with every run.

6.4 Legal and Regional Validity

The GDPR pack maps technical signals to questions motivated by official provisions [1]. It cannot determine controller identity, processing purpose, lawful basis, Article 9 condition, necessity, proportionality, consent validity, data-subject expectations, retention, recipients, safeguards, or jurisdiction. The application’s output is therefore an accountability aid. A qualified reviewer must examine the underlying processing and applicable law.

Adding another regional pack requires more than translation. The maintainer must identify scope, legal authority, effective date, regulated actors, exceptions, remedies, and concepts that have no GDPR analogue. Each pack needs independent legal sign-off, versioned fixtures, source preservation, localisation review, and an update policy. The current Research Baseline pack is not evidence that a second legal regime has been validated.

6.5 Usability and Accessibility

The redesigned interface applies recognised accessibility and usability principles [11–13], and the rendered evidence supports a positive expert assessment. However, conformance-oriented inspection cannot establish comprehension, efficiency, satisfaction, trust calibration, or willingness to continue using the product. No screen-reader campaign, switch-access test, colour-vision simulation, large-font matrix, or participant study was completed. These are required before making an accessibility-conformance or user-adoption claim.

6.6 Interpreting the Engineering Result

The evaluation is strongest when it is read as a chain of bounded claims rather than as a single pass or fail judgement. At the innermost layer, unit and integration tests establish that a specified input produces the specified state transition. At the next layer, release assembly and manifest inspection establish that the tested TypeScript is incorporated into an installable Android artefact with the recorded package properties. Physical-device interaction then establishes that selected routes render and selected controls respond on one hardware and operating-system combination. None of these layers automatically establishes the next one. A correct function may be absent from a package, an installable package may render poorly, and a coherent interface may still be misunderstood by users.

This layered reading prevents a common reporting error in prototype research: promoting evidence because several individually positive checks appear together. Ten successful checks are not equivalent to one check of a broader proposition. For example, a passing threshold test and a successful device launch do not jointly prove that the threshold represents unlawful processing. They establish two different facts, one logical and one operational. Privacy Lens therefore records evidence by claim type. The resulting report may look more cautious than a conventional demonstration, but it is more useful to a maintainer because it shows exactly which additional study could change a conclusion.

The phrase “initial store-submission engineering candidate” is deliberately narrow. It means that the repository can produce the expected release formats, that the product has a coherent user journey and disclosures, and that no known in-scope engineering defect makes controlled submission work irrational. It does not mean that the application is signed with the owner’s production key, that a store has reviewed its declarations, or that the legal mappings have been approved by counsel. Separating these meanings is particularly important for privacy software because a polished security or compliance interface can borrow authority from the domain even when its evidence is limited.

There is also a difference between absence of an observed failure and affirmative evidence of a desirable property. No crash during a short device session is evidence about that session; it is not positive evidence of long-term stability. A small memory range across a few restarts is a useful anomaly check; it is not a leak proof. No sensitive runtime permission in the merged manifest is a strong package fact, but it does not by itself prove that every future acquisition connector will remain privacy preserving. The evaluation uses explicit negative claims to preserve these distinctions.

The result should therefore be understood as a reproducible decision point. A future reviewer can rebuild the same revision, inspect the same rules and fixtures, and ask whether later evidence warrants promotion from one claim class to another. That property is more durable than a one-time declaration of readiness. It converts the thesis from a retrospective narrative into a maintainable assurance record.

6.7 Acquisition Authority and Observation Quality

The largest external-validity constraint is not the visual interface or the threshold arithmetic; it is the authority and completeness of the observation source. Android deliberately limits one ordinary application’s access to sensitive information about other applications. Privacy Lens does not bypass that security model. The current native bridge and imported-record contracts can carry observations when an authorised deployment supplies them, while the simulator supports deterministic explanation and testing. The prototype must not imply that installing it from a public store grants unrestricted historical visibility.

This constraint affects every downstream conclusion. An apparently quiet history may mean that accesses did not occur, that the selected API does not expose them, that an OEM shortened retention, that the collection job was delayed, or that the current deployment lacks authority. These explanations are not interchangeable. The evidence schema therefore includes source and time-window fields, and an evidence-gap state is preferred when acquisition sufficiency is unknown. A “no technical concern” result refers only to accepted records, not to all activity on the device.

Observation quality has at least five dimensions. Coverage asks which applications, operations, and time intervals are visible. Resolution asks whether records are event-level or aggregated. Timeliness asks how long after an event a record becomes available. Integrity asks whether records can be altered, replayed, or reordered. Semantic fidelity asks whether the operation label actually corresponds to the personal-data behaviour under review. A future connector should publish a capability document addressing all five dimensions. Merely conforming to the TypeScript shape is insufficient.

Aggregate records are attractive because they reduce storage and may reduce the privacy footprint of the reviewing application. They also erase context. Ten accesses could represent ten user-requested map refreshes or repeated background tracking; a total cannot distinguish them. Conversely, event-level traces can support better interpretation but may themselves reveal sensitive routines, application use, or location-related behaviour. The acquisition design must therefore minimise collected fields, bound retention, protect local storage, and explain who is authorised to inspect the evidence. More data is not automatically a more privacy-preserving audit.

An authorised enterprise or research deployment could improve coverage through device-owner controls, instrumented test applications, or controlled firmware. Each option changes the population to which the findings apply. Device-owner evidence may be relevant to managed fleets but not to ordinary consumers. Instrumented application evidence can establish ground truth for that application but not for unrelated packages. Research firmware may provide

high-resolution traces but changes the operating environment. Evaluation reports should name the authority model rather than treating all Android evidence as equivalent.

Future acquisition validation should pair the connector output with an independent event generator. The generator should produce timestamped, permission-specific actions under known foreground and background conditions. The study should compare generated and acquired events, report missing and duplicate records, measure timestamp error, and repeat the process across Android versions and OEMs. This would create an empirical coverage statement. Until then, the architecture supports authorised evidence, but the public-store product remains primarily a review and demonstration layer.

6.8 Threshold Meaning, Calibration, and Decision Error

The implemented baselines and multipliers are transparent prototype parameters. Transparency makes the rule reproducible; it does not make the parameters representative. A location threshold of thirty-six accesses in a twenty-four-hour record can be inspected and tested exactly, but its normative meaning depends on application category, user action, service duration, and declared purpose. A navigation application and a static calculator cannot be judged by the same behavioural expectation merely because both run on Android.

False positives and false negatives also have asymmetric consequences. A false positive may alarm a user, damage confidence in a legitimate application, and contribute to warning fatigue. A false negative may reassure the user or reviewer despite activity that deserves investigation. The appropriate balance is not a purely mathematical optimisation because the costs depend on use context. The current design therefore avoids converting a below-threshold result into a compliance statement and avoids converting an exceedance into a violation verdict.

Empirical calibration would require a sampling frame, not simply a large collection of counts. Applications should be stratified by category, operating mode, permission purpose, foreground status, and user task. Collection periods should capture weekdays, weekends, intermittent use, and long sessions. The study should document missing observations and OEM-specific retention. Independent reviewers would need an annotation protocol for deciding which cases warrant investigation, and disagreement should be reported rather than removed through silent consensus.

The calibration target should also be explicit. Optimising agreement with human reviewers is different from predicting store-policy enforcement, detecting malware, or identifying GDPR accountability questions. Each target requires different labels and evidence. Privacy Lens targets review prioritisation: the desired output is a defensible prompt for further examination. That target supports conservative wording and structured missing-context questions rather than a binary compliance classifier.

Multi-scale features improve coverage but add parameters. The daily excess rule identifies sustained volume, the burst rule identifies concentrated activity, and cross-window logic identifies repeated near-threshold behaviour. These signals may correlate, so simply summing them could exaggerate risk. A future scoring design should explain whether each signal contributes independently, whether one supersedes another, and how uncertainty changes when

source resolution is coarse. It should also preserve the raw contributing values so a reviewer can reconstruct the decision.

Parameter governance is as important as parameter selection. Every released pack should identify its effective version, parameter owner, change rationale, fixtures, and compatibility requirements. A threshold change can alter stored findings without any change in observed activity. The application should therefore retain the pack version used for a historical decision and should not silently relabel old evidence when a new pack becomes active. Re-evaluation, if offered, should create a new result linked to the prior one.

A responsible calibration programme would begin with controlled application traces and explicit scenarios, progress to a consented multi-device observational study, and only then consider population thresholds. Performance should be reported by category and source capability, with confidence intervals and error analysis. Even strong empirical performance would support prioritisation accuracy, not legal adjudication. That claim boundary should remain invariant as the data set grows.

6.9 Temporal Evidence and Replay Semantics

Time-windowed evidence introduces failure modes that are easy to overlook in static test cases. Two records can describe the same underlying accesses, adjacent periods, partially overlapping periods, or unrelated periods with identical totals. If overlapping aggregates are added, one event may be counted twice. If late records move a retention watermark backwards, replayed evidence can alter later decisions. If future timestamps are accepted without tolerance, clock errors can create impossible histories.

The current engine addresses these risks with safe integer checks, a maximum window duration, bounded future tolerance, replacement of identical windows, a finite per-key history, and non-overlap requirements for cross-window evidence. These are meaningful controls because they specify how the repository behaves under replay. They do not create event-level identity where none exists. Partially overlapping aggregates cannot always be reconciled without access to their constituent events.

The conservative response is to decline cumulative interpretation when independence cannot be established. This may reduce sensitivity, but it avoids manufacturing precision. A future event-level connector could attach stable event identifiers or monotonic collection cursors. A future aggregate connector could publish disjoint collection intervals and a watermark signed or otherwise protected by the producer. Both designs would strengthen deduplication, but each must account for device clock changes and interrupted collection.

Retention limits deserve separate scrutiny. Keeping only the most recent records bounds memory and reduces local exposure, yet an arbitrary truncation point can remove context needed for a decision. The history limit of 4,096 entries is an engineering safeguard, not a legal retention determination. Production policy should relate retention to the stated review purpose, source frequency, device capacity, and user deletion expectations. The Settings clear action should remove review data predictably, while audit exports or externally managed evidence require their own lifecycle.

Temporal testing should cover more than threshold boundaries. Useful properties include idempotence under exact replay, invariance to arrival order for independent windows, rejection of unsafe durations, monotonic retention watermarks, bounded storage, and stable decisions after process restart. Tests should also inject clock rollback, daylight-saving changes, long device sleep, and records delivered after the active pack changes. These cases would reveal whether the implementation depends on wall-clock assumptions that are not documented.

The broader lesson is that time is part of the evidence contract, not metadata attached after classification. A risk statement about repeated behaviour is only as defensible as the window relationships used to compute it. Making those relationships visible helps reviewers distinguish sustained behaviour from an artefact of aggregation.

6.10 Regulation-Pack Governance Beyond Software Modularity

The replaceable pack contract solves a software architecture problem: it prevents jurisdiction-specific references, prompts, and thresholds from being scattered across screens, storage, and native integration code. It does not solve the institutional problem of deciding what a pack should say. A pack can be syntactically valid and legally misleading. Consequently, extensibility must be paired with admission and maintenance governance.

Admission should begin with scope. The pack must name the jurisdiction, regulated context, legal instruments, effective date, intended users, and exclusions. It should distinguish binding provisions from regulator guidance and from research heuristics. Every rule should link a technical condition to a review question and explain the missing facts that prevent automated legal determination. A reviewer should be able to trace displayed wording to the pack source and from the source to the governing material.

Legal review should be independent of the developer who encoded the rule. The reviewer should assess citation accuracy, applicability, exceptions, terminology, translations, and the risk that interface wording implies a verdict. Approval should apply to a specific pack version and date. Where the law or guidance changes, the registry should deprecate the old version, explain the transition, and preserve historical identity for prior findings.

Technical admission should then validate identifiers, semantic versions, rule uniqueness, supported evidence fields, bounded numerical parameters, localisation completeness, and fixture coverage. The application should reject an unknown or malformed pack rather than fall back silently to a default. If a selected pack is unavailable after an update, the interface should present an explicit blocked state and retain the user's selection for recovery.

Regional variation is not merely a change of article numbers. Concepts such as controller, covered business, sensitive information, consent, sale, sharing, child, or data localisation may not map one-to-one. Enforcement actors and private rights may differ. A common interface model should therefore contain generic evidence and review primitives while allowing packs to declare jurisdiction-specific questions. Forcing every regime into GDPR terminology would create superficial flexibility and substantive error.

Localisation also requires more than translating strings. Legal terms may have authoritative language versions, while plain-language prompts need comprehension testing in the user's

locale. Text length affects layout, and right-to-left scripts affect navigation and icon direction. A pack release should therefore combine legal-language review, plain-language review, and rendered interface testing. The interface should identify both the selected region and pack version so the user understands which framework generated the prompt.

Distribution is another trust boundary. If packs are ever downloaded, the system will need authenticity, integrity, rollback, and availability controls. Signed manifests, pinned publisher identities, checksum verification, and an offline last-known-good version are plausible mechanisms. Remote updates should not introduce executable code; the contract should remain declarative and narrowly validated. The present bundled registry avoids network update risk, which is appropriate for the current candidate.

The Research Baseline pack illustrates an important disclosure pattern. It proves that the product can switch rule sets and preserve identity, but it intentionally does not masquerade as a validated regional law. Its name and explanatory text should remain visibly non-legal. A future second jurisdiction should only be advertised after completing the same source, review, fixture, localisation, and update-governance process as the GDPR pack.

6.11 Trust, Comprehension, and Willingness to Use

Visual coherence is necessary because users make rapid judgements about whether a privacy tool is maintained and safe. Consistent typography, restrained colour, stable navigation, and plain-language hierarchy reduce avoidable friction. They are not sufficient for trustworthiness. A visually polished product can be more dangerous if polish causes users to accept unsupported conclusions. Privacy Lens therefore treats calibrated trust, rather than maximum trust, as the design objective.

Calibrated trust means that confidence changes with evidence quality. A native or authorised imported observation can be described according to its documented capability. A simulator record must remain labelled synthetic. Missing purpose or lawful-basis context must remain visible even when the numerical signal is normal. The interface should make the most important limitation available at the decision point, not bury it exclusively in a policy page.

Willingness to use has several components. Initial adoption depends on a credible value proposition and low setup burden. Continued use depends on whether findings arrive at useful times, whether repeated alerts are suppressed, whether explanations support action, and whether local data controls behave predictably. Recommendation to others may depend on institutional endorsement, independent review, and evidence that the tool does not create new privacy risks. A short expert inspection can assess obvious barriers but cannot establish these behavioural outcomes.

A participant study should therefore use realistic tasks rather than preference ratings alone. Participants could identify the source of a finding, compare two risk states, explain why a low-count record is not a compliance certificate, switch regulation packs, clear local evidence, and decide what follow-up information is needed. Measures should include completion, error, time, comprehension, perceived workload, appropriate reliance, and confidence. Interviews can reveal whether terms such as evidence gap or technical concern match users' mental models.

The sample should include ordinary Android users, privacy-aware users, accessibility participants, and professional reviewers. Their goals differ. Ordinary users may need a concise decision path; professionals may need exportable provenance and exact rule details. Accessibility participants may expose focus order, verbosity, contrast, magnification, or motor-control barriers that visual review misses. The study should avoid treating one group's preference as universal.

Longitudinal testing is necessary for notification design. A single session cannot reproduce fatigue or habituation. A multi-week study could compare immediate alerts, daily summaries, and state-change-only notifications. It should record dismissals and follow-up actions while avoiding collection of unnecessary behavioural telemetry. The hypothesis is not that fewer notifications are always better, but that each interruption should correspond to a meaningful change.

Trust measures also need a correct-answer component. Asking whether participants trust the application can reward unjustified confidence. Better evaluation presents cases with strong evidence, synthetic evidence, missing context, and malformed input, then tests whether confidence tracks those differences. An effective design should sometimes cause the user to withhold judgement. That is a successful outcome for an evidence-bounded review tool.

6.12 Privacy and Security of the Reviewing Application

A privacy-review application can itself become a source of sensitive data. Permission histories may reveal health routines, mobility, communication, or working patterns even when they contain no content. Application identifiers can expose relationships or interests. Findings and notification text may be visible to other people with physical access to the device. The product's local-first design reduces network disclosure, but local processing is not automatically risk free.

The current package requests no sensitive runtime permission and disables Android backup. These are valuable minimisation facts for the evaluated build. The application also provides a clear-data control and does not require an account. Future connectors could change this profile. Every new acquisition or synchronisation feature should therefore trigger a new data-flow inventory, manifest reconciliation, threat review, privacy notice update, and store declaration review.

At-rest protection should be proportionate to the threat model. Platform sandboxing protects data from ordinary applications, while device compromise, debug builds, rooted environments, and unlocked backups present different risks. If future deployments store event-level histories or legal annotations, encryption and key lifecycle may become appropriate. Encryption does not eliminate exposure through rendered screens, notifications, logs, or exported files, so those channels also require controls.

Logging is a frequent source of accidental disclosure. Production builds should avoid writing raw evidence, application inventories, or legal-context notes to Logcat. Crash reports, if introduced, should be opt-in where required, minimise payloads, document processors and retention, and avoid attaching the evidence database. Analytics should not be added merely

to measure engagement; a privacy tool carries a higher burden to justify behavioural tracking.

Pack supply-chain risk must also be considered. A malicious or compromised pack could present alarming text, suppress review, or direct users to unsafe resources even without executable code. Declarative schema restrictions, signed distribution, trusted publisher identities, URL allow-lists where appropriate, and human review reduce this risk. Bundled packs should be covered by the same source-control and release-review process as application code.

Abuse scenarios include importing fabricated evidence to defame an application, replaying old records as current, selecting a misleading jurisdiction, and showing a screenshot without its source label. Privacy Lens addresses some of these risks through provenance, timestamps, pack identity, and replay controls. It cannot authenticate every external producer in the current prototype. Export formats should preserve those fields and should state that the document is a review artefact rather than a regulator finding.

Security maintenance is part of store readiness. Dependencies, Android target requirements, signing keys, build provenance, and vulnerability disclosures require ongoing ownership after submission. The present thesis records a reproducible snapshot; it cannot guarantee that the same dependencies remain appropriate later. A release process should include dependency review, secret scanning, reproducible build records, and an incident contact before the product is distributed broadly.

6.13 Research Ethics and Responsible Evaluation

The safest current evaluation uses synthetic or controlled evidence because real permission histories can expose third-party and participant behaviour. Synthetic data provides precise ground truth and repeatability, but it must be labelled so screenshots and metrics are not mistaken for field evidence. The simulator label is therefore an ethical as well as methodological control.

A future participant or device study would require informed consent describing what is collected, why it is needed, how long it is retained, who can access it, and how withdrawal operates. Researchers should avoid collecting unrelated application inventories or fine-grained histories when controlled test applications can answer the research question. Test accounts and scripted actions should be preferred for validation of acquisition correctness.

The legal subject matter creates an additional risk of participant misunderstanding. Study materials should explain that findings are prompts and that the research team is not providing individual legal advice. Participants should not be encouraged to confront application developers or change essential device settings solely because of a synthetic warning. Debriefing should clarify which cases were fabricated and how the evidence boundaries work.

Reporting should include failed runs, missing observations, exclusions, and changes made after pilot testing. Selectively presenting only coherent screenshots would exaggerate maturity. The repository's separation of logical, package, physical, and visual evidence provides a structure for ethical reporting because each claim can be tied to its acquisition method.

If external legal experts annotate cases, their independence, jurisdiction, instructions, and disagreement process should be recorded. A single expert opinion is valuable but should not

be transformed into universal ground truth. Where reasonable interpretations differ, the pack can present the ambiguity as a question rather than forcing a binary label.

Finally, public release creates responsibilities to people who were not research participants. Store text and onboarding must not imply regulatory endorsement. Support channels should handle correction and deletion questions. Pack updates should communicate material interpretation changes. Responsible deployment is therefore a continuing governance activity, not the final administrative step after technical acceptance.

6.14 Store and Operational Readiness

The repository contains target-SDK configuration, AAB output, privacy-policy content, a stable package identifier, branded assets, and a store checklist. Google Play nevertheless requires owner-controlled signing and console information, including a Data safety declaration and current policy compliance [18, 19]. The present AAB uses a QA signing fallback when no upload-key environment is provided. Public submission therefore remains blocked on an upload key, HTTPS privacy-policy publication, support contact, final listing content, screenshots selected for the listing, console declarations, internal-track testing, and store review.

6.15 Prioritised Next Work

The next programme should proceed in this order:

1. obtain independent GDPR-pack review and define pack-update governance;
2. provision production signing and complete Play Console privacy and Data safety declarations;
3. run accessibility and localisation matrices, followed by task-based participant evaluation;
4. execute multi-OEM, multi-version, 24-hour scheduling, battery, memory, and recovery tests;
5. replace aggregate-only acquisition where authorised event-level evidence is available;
6. design and independently validate one additional regional pack before claiming multi-jurisdiction support.

This order prioritises validity and operational trust over adding more visible features.

7 Conclusion

7.1 Answers to the Research Questions

RQ1: deterministic and safe transformation. Permission summaries are accepted through a conservative evidence contract, rejected when identifiers, numbers, sources, windows,

timestamps, or contexts are unsafe, and evaluated through a deterministic rule-pack engine. Temporal isolation addresses replay and overlap within the limits of aggregate windows.

The answer is supported by more than nominal examples. Runtime validation treats TypeScript declarations as developer guidance rather than as a security boundary. Permission identifiers must be owned values from the admitted set; numeric fields must be safe integers; windows must be ordered, bounded, and plausibly timed; and source values must match the closed provenance vocabulary. A rejected observation does not become a normal finding. This fail-closed behaviour is important because silent normalisation would create reassurance from evidence whose meaning is unknown.

Determinism also includes state. An identical record does not accumulate repeatedly, a late record cannot move the repository’s retention watermark backwards, and cross-window evaluation uses completed non-overlapping history. These controls make repeated test runs explainable and prevent several aggregate replay errors. The qualification remains that aggregates do not contain stable identities for their constituent events. The implementation can refuse unsafe combination; it cannot reconstruct information the producer did not supply.

RQ2: calibrated communication. Each finding preserves provenance, temporal scope, rule-pack identity, rationale, and missing-context questions. The interface distinguishes needs-review, evidence-gap, and no-technical-concern states, and repeatedly states that technical output is not a legal verdict. Synthetic demonstrations remain labelled throughout the data path.

Calibrated communication is implemented as an information architecture, not as a disclaimer appended to an otherwise absolute score. Overview explains the product’s current evidence and offers a clear next action. Findings organises review states and exposes detail progressively. Settings identifies the selected region and provides data controls. Within a finding, status, source, time, numerical reason, pack identity, and follow-up questions appear in an order intended to support both rapid triage and deeper inspection.

The communication result remains an expert assessment of the implemented design. The device render and interaction checks show that the path exists and is visually coherent on the tested handset. They do not prove that a population will understand the distinction between technical concern and legal compliance. A task-based participant study is required to establish that claim. The thesis therefore answers RQ2 at the level of traceable interface design and bounded device evidence, not validated human comprehension.

RQ3: regional extensibility. Jurisdiction-dependent material is loaded from registered, versioned packs behind a stable contract. The EU GDPR pack is the evaluated legal objective; the Research Baseline demonstrates software switching without claiming another jurisdiction. New regional packs require their own legal and empirical validation.

This result separates two forms of extensibility that are often conflated. Software extensibility means that another admitted pack can supply identifiers, labels, thresholds, legal references, rationales, and questions without rewriting navigation, persistence, or Android integration. Substantive legal extensibility means that those contents are correct for a jurisdiction and use context. The project demonstrates the first and defines governance requirements for the

second. It does not use a placeholder pack as evidence of comparative-law validity.

The contract also improves maintenance of the evaluated GDPR objective. A finding stores the pack and rule version that produced it, so a later rule update need not erase historical meaning. Validation rejects malformed packs, while explicit selection avoids hidden regional inference. This design provides a foundation for future jurisdictions, but every production pack still needs scoped sources, independent review, versioned fixtures, localisation, and an update policy.

RQ4: supported claims. Compilation, lint, deterministic and adversarial suites, release APK/AAB assembly, merged-manifest inspection, physical installation, cold launch, and rendered workflows passed for the recorded candidate. These results support an initial store-submission engineering candidate. They do not prove legal compliance, unrestricted Android visibility, long-run reliability, broad usability, accessibility conformance, production signing, or store acceptance.

The answer to RQ4 is therefore a claim map rather than a maturity slogan. Logical suites support engine conformance to specified rules. Package inspection supports statements about identifiers, SDK targets, permissions, backup configuration, and incorporated resources. Physical checks support only the recorded handset, build, and interaction interval. Visual review supports an expert judgement that the candidate is coherent enough for controlled submission work. External owner actions and broader empirical studies remain explicit gates.

This structure makes negative results useful. If a future OEM test misses observations, the acquisition claim can be narrowed without invalidating the regulation-pack architecture. If participants misunderstand a label, the presentation layer can change without rewriting historical evidence. If legal review changes a mapping, a new pack version can be admitted without silently rewriting old findings. The research questions are therefore connected by contracts while retaining separable evidence.

7.2 How the Contributions Fit Together

The first contribution, the evidence-bounded architecture, establishes the governing principle: every conclusion must retain the source, temporal scope, rule identity, and known missing context needed to interpret it. This principle influences data types, validation, storage, interface language, testing, and release documentation. It is not a wrapper around the implementation. Without it, individual technical features could still work while the product as a whole overstated its authority.

The second contribution, the fail-closed decision pipeline, operationalises that principle at the point where data enters the system. A permissive pipeline would transform malformed or inherited values into ordinary results, creating an especially harmful false sense of safety. Closed identifiers, safe numbers, bounded windows, explicit sources, replay semantics, and structured evidence gaps ensure that uncertainty remains visible. The pipeline is intentionally deterministic so an auditor can reconstruct why a record was accepted and why a finding changed.

The third contribution, the regulation-pack contract, locates jurisdictional interpretation

in a reviewable component. This reduces coupling and clarifies the boundary between generic signal processing and legally motivated questions. The contract includes identity and versioning because a rule without provenance cannot support historical accountability. The contribution is the architecture and governance model, not a claim that arbitrary legal systems can be translated automatically.

The fourth contribution, the local-first interface, turns the internal evidence model into a product experience. The three-destination structure removes experimental navigation and emphasises current state, actionable review, and controls. Status is not encoded by colour alone. Synthetic provenance and claim limits are presented near relevant decisions. Local storage, no account requirement, no sensitive runtime permission, disabled backup, and a clear-data action align the product behaviour with its privacy message.

The fifth contribution, the verification record, connects artefacts to claims. Deterministic fixtures, adversarial boundary cases, build outputs, manifest facts, device screenshots, and store-readiness gaps are not collapsed into one test total. This makes the evaluation auditable and limits accidental overstatement. It also defines a repeatable path for future releases: every material change should identify which evidence layers must be renewed.

Together, the contributions support a form of trustworthy incompleteness. Privacy Lens does not solve every acquisition, legal, or usability problem, but it makes those unsolved parts structurally difficult to hide. In an accountability tool, that property is itself useful. A system that clearly refuses an unsupported verdict can better support responsible human review than a more ambitious classifier whose authority is unclear.

7.3 Contributions

The thesis contributes:

1. an evidence-bounded Android privacy-review architecture;
2. a fail-closed, temporally isolated decision pipeline with explicit provenance;
3. a replaceable regulation-pack contract that separates legal mappings from product infrastructure;
4. a local-first, three-destination interface designed for calibrated trust;
5. a reproducible verification record that separates logical, package, physical-device, legal, and deployment claims.

7.4 Final Product Position

Privacy Lens 1.1.0 was judged visually coherent, trustworthy within its disclosed boundaries, and usable enough to invite continued use in a controlled initial-submission context. The final release artefacts use package `com.zhihengzhang.privacylens`, target SDK 36, and request no sensitive runtime permission. They are accompanied by a privacy-policy draft, user guide, product audit, store-readiness checklist, checksums, and physical-device screenshots.

The candidate is not yet a publicly publishable production release. Production signing, stable public policy and support endpoints, console declarations, internal-track evaluation, and store review remain external owner actions. This distinction is part of the result rather than an administrative footnote: trustworthy privacy software must disclose where its evidence ends.

7.5 Deployment and Evidence Roadmap

The first deployment stage should be an owner-controlled internal track. Before upload, the owner should provision a protected upload key, build from a clean tagged revision, record checksums, verify the final merged manifest, and publish stable HTTPS privacy-policy and support pages. The Play Console Data safety answers must be reconciled against the exact binary and any introduced diagnostics. Test accounts are unnecessary for the current local-first design, but the listing should explain that simulated evidence is available for demonstration and that third-party visibility depends on authorised sources.

Internal-track evaluation should renew the functional and visual evidence using the store-delivered build rather than a locally installed sibling. Testers should cover first launch, the core review path, regulation-pack switching, evidence deletion, process death, update from the prior version, offline behaviour, large text, screen reader navigation, and representative screen sizes. Crashes and blocked tasks should be recorded with build identity. The aim is to find operational defects, not to claim population usability from a small group.

The second stage should add a closed participant study focused on comprehension and calibrated reliance. The study should include synthetic, authorised, missing-context, and malformed cases. Participants should explain what the result establishes, identify what is missing, and choose an appropriate next action. Success requires more than positive ratings: users should avoid both unwarranted alarm and unwarranted reassurance. Findings from this stage should drive terminology and hierarchy changes before a wider release.

The third stage should validate acquisition in one explicit authority model. A controlled event generator should establish ground truth across several Android and OEM combinations. Coverage, duplicates, timestamp error, scheduling delay, retention, restart recovery, battery, memory, and failure disclosure should be measured. The listing and in-app source descriptions must reflect the measured capability. If ordinary public-store deployment cannot provide useful external evidence, the product should position itself honestly as an imported-evidence reviewer or research demonstrator rather than imply device-wide monitoring.

The fourth stage should establish regulation-pack governance. Independent reviewers should approve the GDPR pack's citations, terminology, questions, and effective scope. A release process should require source records, fixture changes, localisation review, and deprecation notes. Only after one additional jurisdiction completes the same process should the product claim validated multi-region operation. Until then, the pack switch proves architectural flexibility.

The fifth stage should consider broader distribution. At that point, the maintainer needs vulnerability and dependency processes, incident handling, correction requests, user-support

ownership, pack-update continuity, and periodic store-policy review. Metrics should be collected only when they are necessary and ethically justified. A privacy product should not undermine its value proposition through default behavioural analytics.

Each stage has a promotion criterion. Passing a stage authorises the associated claim and no broader one. This prevents pressure to translate technical progress into legal or market certainty. It also makes deferral acceptable: if production signing or independent legal review is unavailable, the candidate can remain a useful controlled artefact without pretending those gates have been met.

7.6 Lessons for Privacy-Engineering Practice

The first lesson is that provenance must survive every transformation. Capturing a source label at ingestion is insufficient if storage, notifications, exports, or screenshots omit it. Provenance should be treated as part of the value being computed. Tests should therefore follow it from observation through decision and presentation.

The second lesson is that missing context requires a first-class state. Many privacy questions cannot be answered from technical frequency. Returning a neutral or positive result when purpose, authority, or acquisition coverage is unknown creates hidden assumptions. An evidence-gap state makes the limitation actionable: it can ask for a privacy notice, controller record, consent basis, or authorised trace rather than fabricating certainty.

The third lesson is that extensibility needs governance. An interface or schema that can load new legal text is technically flexible but may accelerate error if sources, versions, review, and updates are uncontrolled. The pack boundary is valuable precisely because it creates a unit that can be admitted, tested, rejected, and deprecated.

The fourth lesson is that polished design should reduce ambiguity rather than conceal it. Visual hierarchy can foreground evidence strength, state, and next action. Calm colour and concise language can lower cognitive burden. Yet critical caveats must remain visible. The ethical objective is not to maximise engagement or confidence; it is to help the user reach an appropriately bounded understanding.

The fifth lesson is that store readiness is an evidence package. An AAB is necessary but not sufficient. Signing ownership, manifest reconciliation, data declarations, policy endpoints, screenshots, accessibility, update testing, and support processes all affect whether the distributed product matches the evaluated one. Treating these as afterthoughts would break the thesis's claim chain at the point of deployment.

The final lesson is that disciplined limitation can increase, rather than reduce, practical value. A reviewer can act on a clearly sourced question, ask for missing evidence, and revisit the decision after a pack update. An unsupported compliance badge offers none of those affordances. Privacy Lens is most credible when it remains specific about what it observed, how it reasoned, and what a human must still decide.

7.7 Future Work

Future work should validate the GDPR pack independently, add a genuinely reviewed regional pack, improve authorised event-level acquisition, test multiple Android implementations over long durations, and conduct accessibility and participant studies. These activities should retain the Revision 10 claim-to-evidence discipline. Additional features are valuable only when their authority, provenance, and validation remain visible.

The programme should also investigate formal and property-based techniques for evidence admission and replay. Useful properties include idempotence, bounded storage, monotonic watermarks, source isolation, and stability under reordered independent windows. Mutation testing could assess whether the suite detects weakened validation. A separately implemented oracle would reduce the correlated-defect risk of deriving implementation and expected values from one specification.

Interface research should compare alternative presentations of uncertainty. For example, a three-state label, an evidence-strength scale, and a question-led presentation may produce different reliance patterns. The comparison should measure comprehension and decisions, not only preference. Accessibility evaluation should be incorporated from recruitment through reporting rather than added as a final conformance scan.

The pack framework could support signed declarative updates after its threat model is tested. This would require publisher identity, schema versioning, offline recovery, rollback protection, and clear user communication. Remote executable rules would introduce unnecessary risk; the safest extension preserves a narrow validated data contract.

Finally, future scholarship should study the relationship between accountability prompts and organisational action. A technically clear question has value only if users or reviewers can obtain the missing policy, consent, retention, or controller evidence. Research with data-protection practitioners could examine which prompts accelerate review, which create noise, and how findings should integrate with established assessment records without becoming an unreviewed legal decision.

7.8 Closing Statement

The central finding is that privacy accountability on Android is strengthened not by presenting the strongest possible conclusion, but by presenting the strongest conclusion the available evidence actually supports. Privacy Lens operationalises that principle in code, interface, evaluation, and thesis structure.

References

- [1] European Parliament and Council of the European Union, “Regulation (eu) 2016/679 of 27 april 2016,” 2016.
- [2] European Data Protection Board, “Guidelines 4/2019 on article 25 data protection by design and by default,” 2020.

- [3] H. Nissenbaum, “A contextual approach to privacy online,” *Daedalus*, vol. 140, no. 4, pp. 32–48, 2011.
- [4] IQVIA Institute for Human Data Science, “Digital health trends 2021: Innovation, evidence, regulation, and adoption,” tech. rep., IQVIA Institute for Human Data Science, July 2021.
- [5] L. P. et al., “88% of android mhealth apps share data: A systematic assessment of privacy and security of mobile health apps,” *BMJ*, vol. 373, no. n1248, 2021.
- [6] A. M. McDonald and L. F. Cranor, “The cost of reading privacy policies,” *I/S: A Journal of Law and Policy for the Information Society*, vol. 4, p. 543, 2008.
- [7] J. A. Obar and A. Oeldorf-Hirsch, “The biggest lie on the internet: Ignoring privacy policies and terms of service anymore?,” *Information, Communication & Society*, vol. 23, no. 1, pp. 128–147, 2018.
- [8] European Data Protection Board, “Guidelines 03/2022 on deceptive design patterns in social media platform interfaces,” 2023.
- [9] Android Developers, “Permissions on android,” 2026. Accessed 9 August 2026.
- [10] M. Egele, C. Kruegel, E. Kirda, and G. Vigna, “Pios: Detecting privacy leaks in ios applications,” in *Proceedings of the 18th Network and Distributed System Security Symposium (NDSS)*, 2011.
- [11] International Organization for Standardization, *ISO 9241-11:2018 Ergonomics of Human-System Interaction—Part 11: Usability: Definitions and Concepts*. Geneva: ISO, 2018.
- [12] World Wide Web Consortium, “Web content accessibility guidelines (wcag) 2.2,” 2024.
- [13] Android Developers, “Principles for improving app accessibility,” 2026.
- [14] J. Sweller, P. Ayres, and S. Kalyuga, *Cognitive Load Theory*. Springer Science & Business Media, 2011.
- [15] D. Kahneman, *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [16] J. D. Lee and K. A. See, “Trust in automation: Designing for appropriate reliance,” *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [17] Android Developers, “About android app bundles,” 2026. Accessed 9 August 2026.
- [18] Google Play Console Help, “Meet google play’s target api level requirement,” 2026. Accessed 9 August 2026.
- [19] Google Play Console Help, “Provide information for google play’s data safety section,” 2026. Accessed 9 August 2026.